

Apuntes de Optimización Combinatoria.

(Modelado y Resolución de Problemas de Organización Industrial)

Métodos Cuantitativos de Organización Industrial

José Pedro García Sabater

Grupo de Investigación ROGLE

Departamento de Organización de Empresas

Curso 13 / 14

1	OPTIMIZACIÓN COMBINATORIA. INTRODUCCIÓN	6
1.1	Introducción	6
1.2	Complejidad en la Optimización Combinatoria,	8
1.2.1	Optimización Combinatoria	8
1.2.2	Variaciones (con y sin) Repetición, Combinaciones, Permutaciones.	8
1.2.3	P y NP	9
1.3	Algoritmos básicos.....	10
1.4	Problemas de Optimización Combinatoria.....	10
1.4.1	Según su aplicación.	10
1.4.2	Según su clasificación formal.	12
1.4.3	Según las soluciones que se buscan.	12
1.5	Evaluación de Procedimientos.....	12
2	OPTIMIZACIÓN COMBINATORIA. LA BÚSQUEDA RÁPIDA DE SOLUCIONES	13
2.1	Introducción.	13
2.2	Procedimientos de Resolución Aleatorios.....	14
2.2.1	¿Para qué sirven?	14
2.2.2	¿Pueden funcionar?	14
2.2.3	Un procedimiento de generación aleatoria de soluciones.....	16
2.3	Algoritmos Heurísticos.	16
2.3.1	Concepto de Heurística.....	16
2.3.2	Procedimientos Constructivos.....	17
3	ALGORITMOS ENUMERATIVOS.....	19
3.1	Concepto de Algoritmo Enumerativo.	19
3.2	Algoritmo de Enumeración Completa.....	19
3.3	Estructura de un algoritmo de exploración completa basado en exploración de nodos...20	
3.3.1	Conceptos previos: Nodo	20
3.3.2	Estructura general de un procedimiento de exploración completa basado en nodos.	21
3.3.3	Funciones de evaluación.....	21
3.3.4	Selección del Nodo a explotar.....	22

3.3.5	Generación de nodos (explosión)	22
3.3.6	Eliminación (o cierre) de Nodos.	22
3.4	Otras técnicas de Enumeración basadas en la exploración por nodos.....	23
4	PROCEDIMIENTOS DE MEJORA LOCAL	24
4.1	Vecindario.....	24
4.1.1	Intercambio de 2 elementos	24
4.1.2	2-opt	24
4.1.3	Insertar	24
4.2	Algoritmos de Mejora basados en Vecindario	25
4.2.1	Nomenclatura	25
4.2.2	Mejora Iterativa Simple. Procedimiento.....	25
4.2.3	Descenso Rápido. Procedimiento	25
4.2.4	Mejora Iterativa Aleatorizada. Procedimiento.....	25
5	PROCEDIMIENTOS METAHEURÍSTICOS	26
5.1	Procedimientos de población.....	26
5.1.1	Algoritmos Genéticos	27
5.1.2	Scatter Search y Algoritmos Meméticos	32
5.2	Metaheurísticas de Búsqueda de Entornos	33
5.2.1	GRASP	33
5.2.2	ILS y VNS	34
5.2.3	Recocido Simulado	34
5.2.4	Tabu search.	34
5.3	Metaheurísticas basadas en el reconocimiento de patrones.....	35
5.3.1	Redes Neuronales.....	35
5.3.2	Algoritmo de Colonia de Hormigas.....	35
6	ALGUNOS EJERCICIOS DE OPTIMIZACIÓN COMBINATORIA.....	36
6.1	Problema del FlowShop de 3 máquinas.	36
6.1.1	Descripción del Problema	36
6.1.2	Definición de la estructura de la solución.....	36
6.1.3	Definición del modo de evaluar la solución.	36
6.1.4	Un procedimiento de generación aleatoria de soluciones.....	37

6.1.5	Un procedimiento enumerativo de resolución.....	37
6.1.6	Un procedimiento heurístico.....	37
6.1.7	Un procedimiento de mejora local.....	38
6.1.8	Un algoritmo genético	38
6.2	Problema del viajante de comercio.....	39
6.2.1	Descripción del Problema.	39
6.2.2	Definición de la estructura de la solución.....	39
6.2.3	Definición del modo de evaluar la solución.....	39
6.2.4	Un procedimiento de generación aleatoria de soluciones.....	39
6.2.5	Un procedimiento enumerativo de resolución.....	39
6.2.6	Un procedimiento heurístico.....	40
6.2.7	Un procedimiento de mejora local.....	40
6.2.8	Un algoritmo genético	40
6.3	Problema de Secuenciación JIT	41
6.3.1	Descripción del Problema	41
6.3.2	Definición de la estructura de la solución.....	41
6.3.3	Definición del modo de evaluar la solución.....	41
6.3.4	Un procedimiento de generación aleatoria de soluciones.....	42
6.3.5	Un procedimiento enumerativo de resolución.....	42
6.3.6	Un procedimiento heurístico.....	42
6.3.7	Un procedimiento de mejora local.....	43
6.3.8	Un algoritmo genético	43
6.4	Corte de Piezas rectangulares.....	44
6.5	Quinielas.....	47
6.6	SUDOKU	51
6.7	Secuenciando en la Línea de Montaje	53

1 OPTIMIZACIÓN COMBINATORIA. INTRODUCCIÓN

1.1 Introducción

Como se ha comentado anteriormente el Ingeniero de Organización Industrial diseña y mejora artefactos lógicos. Estos suelen tener forma de herramienta software que interpreta datos en el ámbito de un problema, y los transforma entregando como resultados datos que pueden ser ejecutados.

Específicamente dentro del área de la Gestión de la Producción y las Operaciones, muchos de los problemas que se plantean tienen estructuras especialmente difíciles para garantizar la optimalidad en la resolución.

Entre las herramientas disponibles para generar dichas soluciones, los algoritmos de optimización combinatoria resuelven instancias de problemas que se creen difíciles en general, explorando el espacio de soluciones (usualmente grande) para estas instancias. Los algoritmos de optimización combinatoria logran esto reduciendo el tamaño efectivo del espacio, y explorando el espacio de búsqueda eficientemente.

La mayor parte de los procesos de resolución de problemas en optimización combinatoria no garantizan el óptimo ni siquiera en el contexto de modelo, que no necesariamente es el problema real, pero su aproximación al óptimo es probablemente suficiente.

Los métodos de resolución se pueden clasificar de muchas maneras. Una de ellas permite clasificar los métodos de resolución en 4 grandes grupos:

- a) Algoritmos constructivos: Construyen la solución desde una solución incompleta, a partir de los datos del problema y el conocimiento que del mismo se tiene.
- b) Algoritmos de mejora Comienzan con una solución factible y completa al problema, y esta solución general es modificada gradualmente de un modo progresivo.
- c) Estrategias de "divide y vencerás" En esta estrategia se opta primero por dividir el problema en fragmentos a los que se le aplica cualquier otra estrategia, recomponiendo finalmente la solución.
- d) Estrategias de aprendizaje. Las estrategias de aprendizaje pasan por tomar decisiones, en función de datos conocidos por el resultado de resoluciones anteriores o en la misma resolución.

La clasificación que se utiliza aquí extiende el concepto de heurística tanto para los métodos constructivos como para los de mejora, y considera las últimas dos estrategias las dentro del proceso general de resolución, más que entre los procedimientos de resolución en Optimización Combinatoria.

Los métodos habitualmente utilizados para resolver problemas son del tipo heurístico o metaheurístico. Esto es, son capaces de generar soluciones al problema. Son aproximaciones que pretenden acercarse lo más posible al óptimo, pero que pueden fallar en el intento. En los inicios de la Investigación Operativa, y dada las limitaciones de cálculo automático que entonces se tenía, era habitual desarrollar procedimientos heurísticos (es decir que hallaban soluciones) muy rápidamente. Los

procedimientos eran específicos para cada problema y en general su eficiencia era escasa cuando se intentaban utilizar en otros ámbitos.

Pese a no garantizar la optimalidad, los métodos heurísticos de resolución (incluyendo entre ellos los procesos de mejora local y los algoritmos metaheurísticos), son básicos por varios motivos. En primer lugar son capaces de generar soluciones lo que generalmente es mejor que no tener solución alguna. En segundo lugar se puede decir que alcanzar el óptimo de un modelo que tampoco es exactamente el problema real no es esencialmente grave. Por último ser capaz de diseñar una buena heurística, exige un conocimiento del problema que puede conducir a mejoras de otro tipo.

Con la existencia de ordenadores cada vez más potentes, y aprovechando su capacidad de cálculo han surgido en las últimas décadas, las denominadas técnicas metaheurísticas. Son procedimientos genéricos que pueden ser fácilmente adaptados para resolver

La mayoría de las metaheurísticas tienen como objetivo los problemas de optimización combinatoria, pero por supuesto, se pueden aplicar a cualquier problema que se pueda reformular en términos heurísticos, por ejemplo en resolución de ecuaciones booleanas

Muchos de los procedimientos denominados metaheurísticos se fundamentan generalmente en la capacidad del mundo natural de encontrar buenas soluciones y se intenta reproducir esta capacidad, surgiendo de este modo algoritmos como el Recocido Simulado, los Algoritmos Genéticos, las Redes Neuronales, los Sistemas de Hormigas y GRASP (Díaz, Glover, Ghaziri, Gonzales, Laguna, and Moscato, 1996).

En realidad los fenómenos naturales en los que dicen basarse todas las metaheurísticas tienen en común su capacidad de explorar convenientemente el adyacente posible (Kauffman, 2003) para abandonar óptimos locales.

Se puede realizar una clasificación simplificada de los procedimientos metaheurísticos indicando que hay tres grandes grupos. El primer grupo lo forman los que dada una solución intentan mejorarla (búsqueda local). El segundo grupo lo conforman los que parten de un conjunto de soluciones (población) para intentar obtener una mejor por combinación y evolución de las anteriores. El tercer grupo lo conforman aquellos que pretenden específicamente aprender de soluciones previas.

La búsqueda local es la base de la mayor parte de los métodos heurísticos. Empezando por una determinada solución (en ocasiones generada aleatoriamente) la búsqueda local se mueve hacia un vecino que es mejor que la actual solución de acuerdo con la función objetivo. En general la búsqueda local corre el riesgo evidente de quedar atrapada en un óptimo local y la mayor parte de los algoritmos basados en óptimos locales pretenden evitar este problema.

Estos apuntes se estructuran en dos partes, una dedicada a la introducción a la Optimización Combinatoria y otra a los procedimientos de resolución. Se acompañan los apuntes con resoluciones aproximadas a 3 problemas según serían requeridos en examen.

En la primera parte se pretende, en primer lugar, establecer qué es la Optimización Combinatoria y dar una idea general sobre la complejidad de establecer soluciones óptimas en dichos entornos. Se presenta a continuación una aproximación a los problemas de optimización combinatoria mediante una tentativa de clasificación de los mismos. Se presenta finalmente una aproximación a los criterios que permiten seleccionar algoritmos en función de los requerimientos solicitados.

En la segunda parte se abordan los procedimientos de resolución de problemas de optimización combinatoria clasificados en 3 grandes grupos:

- a) Procedimientos de Resolución Aleatorios
- b) Procedimientos Enumerativos
- c) Procedimientos Heurísticos (se incluyen en estos los Metaheurísticos)

1.2 Complejidad en la Optimización Combinatoria,.

1.2.1 Optimización Combinatoria

Un problema de optimización combinatoria es un problema de optimización en el cual el espacio de soluciones posibles es discreto.

La optimización combinatoria es una rama de la optimización de las matemáticas aplicadas fuertemente relacionada con la investigación operativa, la teoría algorítmica y la teoría de la complejidad computacional. Los algoritmos de optimización combinatoria resuelven problemas que se creen difíciles en general, por la vía de explorar el, habitualmente grande, espacio de soluciones del citado problema. Los buenos algoritmos de optimización combinatoria lo logran por la vía de reducir el tamaño efectivo del espacio a buscar y explorando de modo eficiente dicho espacio.

1.2.2 Variaciones (con y sin) Repetición, Combinaciones, Permutaciones.

Es relevante recordar algunos conceptos de combinatoria, pues estos permiten establecer la dimensión de alguno de los problemas aquí tratados.

Los problemas de combinatoria se pueden definir como la “Selección de un subconjunto de entre un conjunto de elementos con permitiendo (o no) la repetición y siendo (o no) relevante el orden de los mismo”.

Si el subconjunto seleccionado es todo el conjunto original el problema tratado es de permutaciones. Si el orden no importa el problema es de Combinación.

A continuación se recuerdan algunas fórmulas básicas que permiten calcular el número de alternativas posibles a determinadas estructuras

Variaciones sin Repetición: Número de modos de seleccionar y ordenar r elementos de un conjunto

de n , donde ninguno se puede repetir. $A_n^r = \frac{n!}{(n-r)!}$

Variaciones con Repetición: Número de modos de seleccionar y ordenar r elementos de un conjunto de n . n^r

Permutación sin Repetición: Número de modos en el que se puede ordenar un conjunto de n elementos distintos entre sí. $n!$

Permutación con Repetición: Número de modos de ordenar un conjunto de n elementos donde hay m_1 elementos de tipo 1, m_2 elementos de tipo 2... m_S elementos de tipo S . $\frac{n!}{m_1! \cdot m_2! \cdot \dots \cdot m_S!}$

Combinaciones sin Repetición: Número de modos de seleccionar r elementos de un conjunto de n , donde ninguno se puede repetir. $C_n^r = \frac{n!}{r! \cdot (n-r)!}$

Combinaciones con Repetición: Número de modos de seleccionar r elementos de un conjunto de n , permitiendo la repetición en la selección. $C_{n+r-1}^r = \frac{(n+r-1)!}{r! \cdot (n-1)!}$

Por su interés se destacan algunas otras relaciones que permiten calcular de cuantos modos se pueden introducir elementos (iguales o distintos-ordenados y no ordenados) en casillas (iguales o distintas – ordenadas y no ordenadas). Cuando se indica que los elementos están ordenados hace referencia a que interesa el orden, no a que estén previamente ordenados.

r Objetos distintos y no ordenados en n casillas distintas y no ordenadas n^r

r Objetos idénticos en n casillas distintas y ordenadas $C_{n+r-1}^r = \frac{(n+r-1)!}{r! \cdot (n-1)!}$

r Objetos distintos y ordenados en n casillas distintas y ordenadas $C_{n+r-1}^r = \frac{(n+r-1)!}{(n-1)!}$

Todas estas relaciones (junto con otras muchas y –lo que es más importante- el modo de llegar a ellas), se pueden encontrar en (Kauffmann, 1971).

1.2.3 P y NP

Teoría de la Complejidad

La teoría de la complejidad computacional es la rama de la teoría de computación que estudia los recursos, o coste, de computación requerido para resolver un problema dado.

Este coste se suele medir en terminos de parámetros abstractos como el tiempo y el espacio. Tiempo son los pasos que requiere el problema para ser resuelto y el espacio suele referirse a la cantidad de memoria que utiliza.

Otros recursos que se pueden considerar es el número de procesadores en paralelo que se necesitan para resolver un problema en paralelo. En este caso hay que considerar el tiempo paralizante y el no paralelizante.

La teoría de la complejidad no es la teoría de la “computabilidad” que tiene que ver con la capacidad de resolver un problema, independientemente de los recursos requeridos.

P y NP

La clase de complejidad P es el conjunto de problemas de decisión que pueden ser resueltos por una máquina determinista en tiempo polinomial. Esta clase corresponde a la idea intuitiva de los problemas que se puede resolver de modo efectivo en los peores casos.

La clase de complejidad NP es el conjunto de problemas de decisión que pueden resolverse por una máquina no determinista en tiempo polinomial. Esta clase contiene una gran cantidad de problemas que se pretenden resolver. La principal propiedad de los problemas de esta clase es que sus soluciones pueden ser chequeadas de modo eficiente en tiempo polinomial.

1.3 Algoritmos básicos.

- Ordenar
- Buscar el Mayor (o menor)
- Intercambiar
- Sumar
- Volver

1.4 Problemas de Optimización Combinatoria

En este apartado se describen algunos problemas clásicos de optimización combinatoria. La intención inicial al redactar el apartado era establecer una clasificación, pero esto parece casi imposible, habida cuenta de que se pueden encontrar muchos tipos de clasificaciones en función de diferentes criterios para cada tipo de problema. Así, se ha optado por enumerar (sin ánimo de ser exhaustivos ni exclusivos) diferentes tipos de problemas en función de diferentes aspectos como por ejemplo: su aplicación, la clase teórica a la que pertenecen o las soluciones que se pretenden conseguir. Se han dejado aparte clasificaciones que considerarían aspectos como su dureza en términos de complejidad, o la existencia de algoritmos eficientes para su resolución.

1.4.1 Según su aplicación.

1.4.1.1 Secuenciación

Los problemas de secuenciación son, junto con los de rutas, la aplicación más clásica en optimización combinatoria, tanto por la extensión de su aplicación, como por su facilidad de planteamiento. Aún así el número de problemas diferentes que se puede plantear son muchos.

Definir el orden en el que se deben ejecutar las tareas de un proyecto para minimizar su duración, teniendo en cuenta limitaciones en los recursos, es un clásico problema de secuenciación.

La secuenciación en una máquina hace referencia al orden en que un conjunto de trabajos deben pasar por una máquina. Permite considerar aspectos como la duración, fecha de entrega, la importancia del cliente...

Los problemas de secuenciación de una máquina se extienden teniendo en cuenta máquinas en paralelo y/o máquinas en serie (taller de flujo) y de ahí al taller general. Otra extensión habitual es considerar tiempos de preparación de máquinas. Además el uso de múltiples recursos diferentes y limitados forma parte también de las posibilidades

Si además de definir la secuencia (o incluso asignar la máquina) se tuviera que definir la cantidad a producir se entraría en los problemas que unen la lotificación con la Secuenciación (ELSSP).

La secuenciación con mezcla de modelos en una línea de montaje, pretende establecer el orden en el que un conjunto de productos (pertenecientes a varias familias en diferentes cantidades) deben circular por una línea de montaje.

1.4.1.2 Rutas

Los problemas de rutas tratan de establecer el circuito a recorrer para dar un determinado servicio, ya sea de entrega de recepción o de ambos. El problema clásico denominado del viajante de comercio (TSP) supone visitar una y sólo una vez un conjunto de puntos. A este problema se le pueden añadir variantes como incluir varios viajantes, limitar la capacidad de los camiones o la disponibilidad de tiempo, incorporar ventanas temporales de entrega o recepción, obligar o prohibir determinados tramos, hacerlo en una o varias ocasiones por cada tramo...

1.4.1.3 Corte y Empaquetado

Los problemas de corte son los ligados a reducir el consumo de materia prima que se vende o consume troceada. Pueden ser unidimensionales (vigas), bidimensionales (placas de vidrio, cartón o tela) o incluso tridimensionales como en el corte de mármol. También se pueden considerar variantes como que las piezas a obtener tengan una forma regular, o irregular (por ejemplo retales de tela). O restricciones que obligan a mantener determinadas direcciones de corte, o realizar cortes completos.

Los problemas de Trim son una variante de estos problemas en los que de un rollo (de papel o de acero) se trocean otros rollos de anchos y longitudes diferentes). El problema radica en este caso en establecer los modos en los que los diferentes rollos se van haciendo para minimizar no sólo la materia prima desperdiciada sino también las preparaciones de máquina a realizar.

Los problemas de empaquetado son similares siendo los contrarios, pretenden ubicar en la mínima superficie (o el mínimo volumen) posible un conjunto de partes que son inicialmente diferentes.

1.4.1.4 Horarios

Los problemas de horarios son un problema clásico de optimización de combinatoria que tiene en la gestión docente (aunque evidentemente no es exclusiva de ella) una gran cantidad de aplicaciones: el diseño de horarios de clase, repartos de guardias, calendarios de exámenes, son algunos fácilmente

reconocibles. En la industria se pueden encontrar fácilmente cuando se trata de repartir cargas de trabajo desiguales entre operarios con calendarios laborales.

1.4.1.5 Asignación

Los problemas en los que se asignan recursos a tareas o tareas a agrupaciones son otra clase habitual de problemas. El equilibrado de líneas es un problema muy conocido, pero los problemas de asignación de frecuencias o de generación de equipos multidisciplinarios equilibrados no son menos habituales.

1.4.2 **Según su clasificación formal.**

- El problema de la mochila (uni o multidimensional)
- De máximo (o mínimo) flujo
- Problemas de Asignación Cuadrática
- Graph Partitioning Problem
- Bipartite Drawing Problem
- Cubrimiento, Partición y Empaquetado.
- De camino más corto (o más largo)
- El problema de coloreado de grafos
- Linear Ordering Problem

1.4.3 **Según las soluciones que se buscan.**

En algunos problemas, sólo se tiene interés en encontrar una solución factible (e incluso en algunos casos únicamente se pretende saber si existe una solución). En otros casos el objetivo es que la solución factible encontrada sea suficientemente buena o incluso que sea óptima (es decir que no haya ninguna solución mejor). Por último es posible que se tenga interés en localizar todas las soluciones óptimas o incluso todas las soluciones factibles.

1.5 ***Evaluación de Procedimientos.***

Los algoritmos se pueden medir en función de su capacidad de alcanzar buenos resultados, y del coste (tanto en tiempo como en memoria que requieren) para alcanzar dichos resultados.

Estos criterios se denominan: Eficiencia, Robustez y Bondad.

Al comparar los algoritmos se puede realizar tanto en el tiempo de resolución que emplean como en el resultado que obtienen.

En el caso de comparar los resultados estos se pueden comparar contra el óptimo, contra una cota del óptimo, o contra el resultado de otros algoritmos. Lo que se mide puede ser el comportamiento promedio, el comportamiento en el peor de los casos o el número de éxitos (número de veces que se obtiene el óptimo o el mejor resultado).

2 OPTIMIZACIÓN COMBINATORIA. LA BÚSQUEDA RÁPIDA DE SOLUCIONES

2.1 Introducción.

La mayor parte de los procesos de resolución de problemas en optimización combinatoria no garantizan el óptimo ni siquiera en el contexto de modelo, que no necesariamente es el problema real.

(Foulds 1983) clasifica los métodos de resolución en 4 grandes grupos:

a) Algoritmos constructivos: Construyen la solución desde una solución incompleta, a partir de los datos del problema y el conocimiento que del mismo se tiene.

b) Algoritmos de mejora Comienzan con una solución factible y completa al problema, y esta solución general es modificada gradualmente de un modo progresivo.

c) Estrategias de "divide y vencerás" La esta estrategia se opta primero por dividir el problema en fragmentos a los que se le aplica cualquier otra estrategia, recomponiendo finalmente la solución.

d) Estrategias de aprendizaje. Las estrategias de aprendizaje pasan por tomar decisiones mensuales sobre el modo de resolver, en función de datos conocidos por el resultado de resoluciones anteriores o en la misma resolución.

Sin embargo la clasificación que se va a utilizar en estos apuntes extiende el concepto de heurística tanto para los constructivos como para los de mejora, y las últimas dos estrategias las considera dentro del proceso general de resolución, más que entre los procedimientos de resolución en Optimización Combinatoria.

Así se clasifican en estos apuntes los métodos según los siguientes cuatro tipos:

- a) Generación de soluciones por métodos heurísticos constructivos
- b) Generación de soluciones por métodos enumerativos
- c) Generación de Soluciones por Métodos Heurísticos de Mejora Local.
- d) Generación de Soluciones por Métodos Heurísticos de Población.

Además se incorpora la generación de soluciones por métodos aleatorios, pues aunque son ineficientes en el proceso de búsqueda, pueden ser eficaces, tanto para calibrar métodos cómo, y más relevante para comprobar que se ha entendido bien el problema.

Los métodos habitualmente utilizados para resolver problemas son del tipo heurístico o metaheurístico. Esto es, son capaces de generar soluciones al problema. Son aproximaciones y inacciones que pretenden acercarse lo más posible al óptimo, pero que pueden fallar en el intento.

Pese a su no-optimalidad, muchos de estos métodos son básicos por varios motivos. En primer lugar son capaces de generar soluciones lo que generalmente es mejor que no tener solución alguna. En

segundo lugar se puede decir que alcanzar el óptimo de un modelo que tampoco es exactamente el problema real no es esencialmente grave. Por último ser capaz de diseñar una buena heurística, exige un conocimiento del problema que puede conducir mejoras de otro tipo.

2.2 *Procedimientos de Resolución Aleatorios.*

2.2.1 ¿Para qué sirven?

Diseñar un procedimiento de resolución aleatorio debiera, en primer lugar, ser fácil de hacer. Además debiera permitir generar soluciones factibles (o al menos evaluables) de modo rápido. Dos son los motivos por los que se recomienda comenzar el análisis de cualquier problema con el desarrollo de procedimientos de resolución aleatorios:

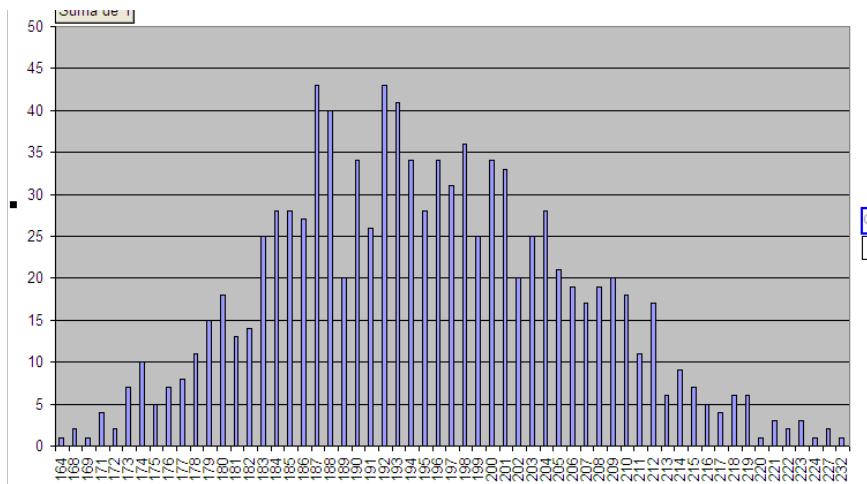
- a) Para comparar contra procedimientos más elaborados.
- b) Para tener rápido un procedimiento que dé soluciones y comprobar que se está abordando el problema adecuado.

Además las soluciones obtenidas aleatoriamente pueden servir de base para otros procedimientos más elaborados (mejora local o algoritmos genéticos).

2.2.2 ¿Pueden funcionar?

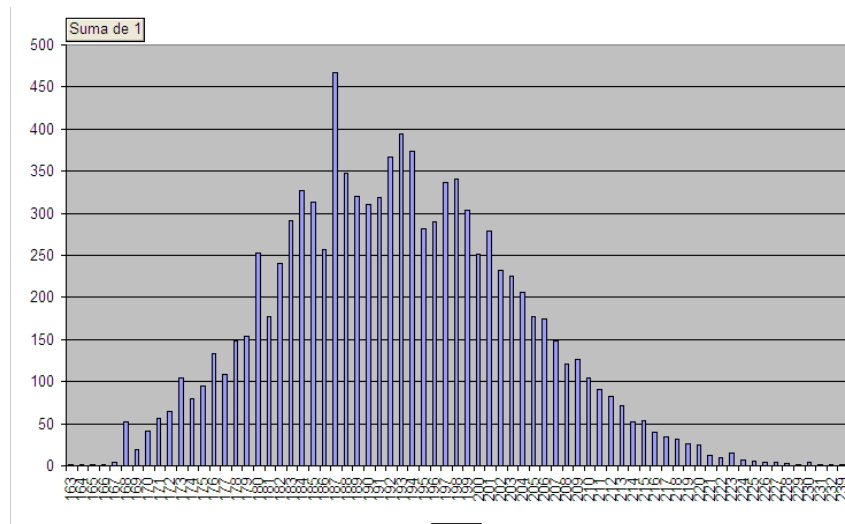
En principio esto dependería de cada problema, pero se puede generalizar diciendo que los resultados aleatorios tienen una baja probabilidad de ser buenos, pero en ese caso, la repetición suficiente podría llegar a dar, de un modo ni eficiente ni robusto resultados aceptables.

Como ejemplo se ha evaluado un problema de Flow-Shop de 3 máquinas, 12 tareas, con tiempos aleatorios de ejecución entre 2 y 20. Se ha ejecutado un algoritmo aleatorio 1000 veces y se ha obtenido la siguiente gráfica de resultados.



Tras 1000 iteraciones en una ocasión se ha obtenido un resultado igual a 164, siendo el óptimo 163 (aunque eso no es conocido a priori).

Se ha repetido el experimento con 10000 iteraciones obteniendo los siguientes resultados.



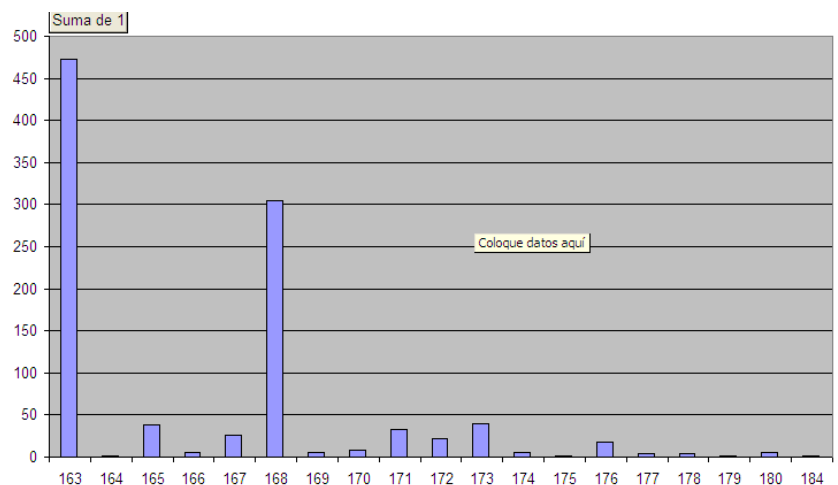
Es decir, en dos ocasiones (de entre 10000) se ha alcanzado el óptimo, y sólo en 10 ocasiones el error cometido sobre el óptimo ha sido menor que el 2,5%.

En problemas más grandes la dispersión crece, y además el tiempo de ejecución no es despreciable.

Sin embargo, y en el mismo problema, con un algoritmo de mejora iterativa sencillo, a partir de una solución aleatoria, se obtiene el óptimo en el 50% de las ocasiones, en la misma cantidad de tiempo que se tarda en generar aproximadamente 100 soluciones aleatorias.

Para comprobarlo se han utilizado las 1000 soluciones aleatorias generadas en la primera experiencia y a cada una de ellas se le ha aplicado un algoritmo de mejora iterativa con un vecindario definido según un intercambio de 2 (común para todos los problemas de permutación).

El histograma resultado ha sido el siguiente:



Es decir, el óptimo se alcanzó en casi el 50% de las ejecuciones y el peor resultado obtenido estaba a 13% del óptimo.

Con el anterior ejemplo se ha pretendido mostrar que las heurísticas aleatorias no funcionan de modo eficiente, aunque son fundamentales, pues permiten iniciar los procesos tanto de modelado como incluso de resolución.

2.2.3 Un procedimiento de generación aleatoria de soluciones

En este apartado se presenta un sencillo procedimiento que permite generar soluciones aleatorias para problemas en los que el vector solución es una permutación de n elementos.

Sea v un vector con n elementos a permutar

```
i:=0;
while i<n-1 do begin
  j:=random(1..n-i)
  intercambiar(v[i+1];v[i+j])
  i:=i+1
end;
```

Intercambiar es un procedimiento que intercambia los valores de v en las posiciones señaladas.

```
Procedure intercambiar(v[i+1];v[i+j])
begin
  aux:=v[i+1];
  v[i+1]:=v[i+j];
  v[i+j]:=aux;
end;
```

2.3 Algoritmos Heurísticos.

2.3.1 Concepto de Heurística

Un algoritmo heurístico es un procedimiento que permite encontrar una solución y que suelen diseñarse de modo específico para cada problema. En métodos matemáticos, heurística suele hacer referencia a un procedimiento que busca una solución aunque no garantiza encontrar la mejor solución. En Inteligencia Artificial se suele denominar función heurística a aquella que dirige la búsqueda (o construcción) de una solución, utilizando algún mecanismo más o menos sencillo.

Un buen algoritmo heurístico debe ser eficiente, bueno y robusto. Esto es: debe requerir un esfuerzo computacional realista, su resultado debe estar suficientemente cerca del óptimo, y la probabilidad de obtener una mala solución debe ser baja.

Un tipo especial de heurísticas son las constructivas, es decir aquellas que en cada paso añaden un elemento más a una solución que no ha sido completamente construida. Las Heurísticas constructivas se pueden utilizar siempre que el problema se puede resolver mediante decisiones sucesivas. (Rutas, Secuenciación, Líneas de Montaje...). En muchas ocasiones el objeto de la decisión es evidente, pero no necesariamente es así.

El otro tipo principal de heurísticas son las de búsqueda de entornos o de mejora local. Estas parten de una solución cualquiera (no necesariamente aleatoria) y avanzan buscando en el vecindario más próximo, produciendo mejoras hasta que alcanzan un punto donde ningún elemento en el vecindario es mejor que la solución de la que ya se dispone.

Las heurísticas también pueden ser de tipo voraz (en inglés *greedy*) estas heurísticas eligen siempre las soluciones mejores de modo local para generar el resultado. Algunas heurísticas *greedy* son óptimas (como ejemplo el algoritmo de Johnson para un flow-shop de 2 máquinas), pero no necesariamente es así. Los algoritmos de tipo voraz suelen ser muy rápidos pues no consideran alternativas.

La alternativa a las heurísticas *greedy* son las Heurísticas con *BackTracking*. Estas son heurísticas que si alcanzan un punto de no retorno (o no suficientemente buenos) retroceden en el proceso de reconstrucción, para analizar caminos abandonados.

- Métodos Constructivos
 - ☐ Voraces
 - ☐ Heurísticas con BackTracking
- Heurísticas No Constructivas
 - ☐ Exploración de Entornos
- Métodos Combinados

2.3.2 Procedimientos Constructivos

Las Heurísticas constructivas se pueden utilizar siempre que el problema se puede resolver mediante decisiones sucesivas. (Rutas, Secuenciación, Líneas de Montaje...). En muchas ocasiones el objeto de la decisión es evidente, pero no necesariamente es así.

Aunque cada heurística constructiva es diferente, y no hay ningún límite en su diseño, algunas de las alternativas que permiten plantear heurísticas son las siguientes:

- a) **Reglas de Prioridad:** Se asignan valores a los objetos que se van a seleccionar en el proceso de construcción y se utilizan para elegir la siguiente opción. Los valores pueden calcularse de una vez para el resto del cálculo o variar dinámicamente tras cada decisión dando lugar a las Reglas Estáticas o a las Dinámicas. En muchos casos es importante que la regla de prioridad tenga en cuenta criterios de desempate.
- b) **Dirección de Construcción:** En ocasiones es posible identificar una dirección de construcción. En general se suele trabajar hacia adelante, pero también es posible diseñar heurísticas que funcionen hacia atrás o incluso en las dos direcciones simultáneamente.
- c) **Técnicas de Reducción Previa:** El análisis previo del problema puede llevar a tomar decisiones que afecten a la estructura de la solución antes de comenzar el proceso de resolución. (Por ejemplo eliminar subsecuencias antes de empezar por ser demasiado caras)
- d) **Heurística de Paso Múltiple:** Se repite el cálculo cambiando las reglas de decisión o incluso tomando decisiones de modo estocástico.

- e) **Heurísticas MultiStart:** Repetir la heurística pero comenzando cada vez por un elemento diferente.

2.3.2.1 Miopía de las Heurísticas Constructivas

Uno de los principales problemas de las heurísticas constructivas es la denominada “miopía”. Ésta consiste en el mal comportamiento de algunas heurísticas, según el cual las decisiones tomadas en cada etapa, aunque buenas según el criterio de selección, perjudican la solución final.

Dos son los modos de solución a este problema: dotar de profundidad a la regla de prioridad o diseñar heurísticas con backtracking.

Dotar de profundidad a la regla de decisión es que en cada etapa de decisión se tenga en cuenta, en la medida de lo posible como afectará en las siguientes etapas de decisión la decisión actual. La regla de prioridad en este caso evaluaría no sólo el efecto de la decisión a tomar sino también las decisiones que se tomarían como consecuencia de la que se tome.

Dotar a las heurísticas de técnicas de backtracking. Es decir dotar de capacidad a las heurísticas de volver atrás en las decisiones (y tomar nuevas decisiones) si la situación no es la deseable.

2.3.2.2 Heurísticas con BackTracking

Las heurísticas con backtracking buscan una solución probando una de entre varias alternativas, cada decisión se almacena de tal modo que si se puede demostrar que una decisión fue incorrecta se restablezca la solución en ese punto. Conceptualmente un algoritmo de backtracking es equivalente a una búsqueda en profundidad en un algoritmo enumerativo, donde cada decisión equivale a un nodo en el árbol.

2.3.2.3 Procedimientos de enumeración incompletos.

Como se ha dicho la Optimización Combinatoria es una rama de la Optimización de las matemáticas aplicadas en el cual el espacio de soluciones posibles es discreto. Es decir, el óptimo se podría alcanzar mediante la enumeración de todas las soluciones, aunque esta posibilidad está restringida a tamaños muy pequeños de problema.

Las técnicas de enumeración pueden ser utilizadas para diseñar procedimientos constructivos que sean más rápidos aunque menos eficaces reduciendo el número de nodos a explorar o limitando el tiempo de ejecución.

2.3.2.4 Heurísticas Constructivas. Estructura General y Alternativas de Diseño

Las Heurísticas constructivas se pueden utilizar siempre que el problema se puede resolver mediante decisiones sucesivas. (Rutas, Secuenciación, Líneas de Montaje...). En muchas ocasiones el objeto de la decisión es evidente, pero no necesariamente es así.

Se consideran heurísticas voraces aquellas que avanzan sin posibilidad de deshacer las decisiones tomadas.

3 ALGORITMOS ENUMERATIVOS

3.1 Concepto de Algoritmo Enumerativo.

Un algoritmo Enumerativo o Completa exacto es aquel que garantiza la obtención de la mejor solución posible, el óptimo, por la vía de explorar el espacio de soluciones en su totalidad.

En el problema citado para ejemplificar la búsqueda aleatoria (el Flow-Shop con 12 tareas) el número de alternativas posibles es de 479.001.600. Si en generar y evaluar cada solución se tardará una milésima de segundo el problema requeriría más de 5 días de computación si se pudiera hacer de un modo ordenado y sistemático (de un modo aleatorio sería sencillamente imposible).

A la vista de las anteriores magnitudes es evidente que los métodos de enumeración completa no son prácticos en la mayor parte de situaciones normales. Sin embargo son una buena estructura para comenzar la búsqueda de soluciones.

Los procedimientos de Enumeración Completa se pueden clasificar en ciegos o guiados según la forma de dirigir la exploración. Según si buscan una solución exacta o se conforman con una buena solución los procedimientos son exactos o heurísticos. También se pueden clasificar según si buscan una solución óptima, todas las soluciones óptimas o todas las factibles.

Por último los algoritmos pueden utilizar la Fuerza Bruta analizando una por una todas las soluciones, o pueden tener algún tipo de inteligencia que permita podar aquellas ramas del árbol de exploración que no van a ser útiles.

3.2 Algoritmo de Enumeración Completa

Los algoritmos de enumeración completa son diferentes en función de la estructura de la solución. Así por ejemplo no es lo mismo hacer enumeración completa para soluciones que se representan como un vector de n dimensiones donde los valores pueden ser 0 o 1, que para una solución que se representa como una permutación de valores, o una solución cuya estructura sea una combinación de n elementos tomados de r en r .

A continuación se presenta un algoritmo de enumeración completa para problemas de permutación basado en una propuesta de Dijkstra citado en http://www.cut-the-knot.org/do_you_know/AllPerm.shtml.

```
For c = 1 To M
  i = N - 1
  Do While (S(i - 1) >= S(i)): i = i - 1: Loop
  j = N
  Do While (valor(j - 1) <= valor(i - 1)): j = j - 1: Loop
  Intercambia S(i - 1) con S(j - 1)
  i = i + 1
  j = N
  Do While (i < j)
    Intercambia S(i - 1) con S(j - 1)
    i = i + 1
    j = j - 1
  Loop
Next c
```

El algoritmo propuesto anteriormente enumera de modo lexicográfico a partir de una solución inicial expresada en el vector S .

El que se expresa a continuación enumera combinaciones (r,n) de la página 231 de Matemáticas discreta Escrito por Richard Johnsonbaugh, Óscar Alfredo Palmas Velasco

Salida: Todas las r -combinaciones de $\{1, 2, \dots, n\}$ en orden lexicográfico creciente.

```
1.  procedure combination( $r, n$ )
2.    for  $i := 1$  to  $r$  do
3.       $s_i := i$ 
4.    print  $s_1, \dots, s_r$  // se imprime la primera  $r$ -combinación
5.    for  $i := 2$  to  $C(n, r)$  do
6.      begin
7.         $m := r$ 
8.         $max\_val := n$ 
9.        while  $s_m = max\_val$  do
10.         // se determina el elemento más a la derecha, que no tenga su máximo valor
11.         begin
12.            $m := m - 1$ 
13.            $max\_val := max\_val - 1$ 
14.         end
15.         // se incrementa el elemento más a la derecha
16.          $s_m := s_m + 1$ 
17.         // el resto de los elementos son los sucesores de  $s_m$ 
18.         for  $j := m + 1$  to  $r$  do
19.            $s_j := s_{j-1} + 1$ 
20.         print  $s_1, \dots, s_r$  // se imprime la  $i$ -ésima combinación
21.       end
22.    end combination
```

3.3 Estructura de un algoritmo de exploración completa basado en exploración de nodos.

3.3.1 Conceptos previos: Nodo

Se puede plantear que los procedimientos de exploración siguiendo un sistema arborescente de nodos que se se evalúan, se podan, se seleccionan y se explotan.

Cada nodo hace referencia a una solución parcial (o relajada) del problema. Dicha solución parcial puede ser valorada, por lo que ya se sabe de la solución, o por lo que las futuras incorporaciones a la solución incorporarán.

Cada nodo, excepto el nodo final, es descendiente de algún nodo padre, al que completa parcialmente la solución que aquel traía de sus antecedentes.

Por la vía de la explosión, se generan descendientes a cada nodo se completa un poco más la solución, añadiéndole un subconjunto de opciones que permiten definir un poco más la información.

3.3.2 Estructura general de un procedimiento de exploración completa basado en nodos.

La estructura propuesta parte de la estructura básica nodo que mantiene toda la información relativa a los nodos (como mínimo: cual es el nodo padre, evaluación, que información incorpora a la solución del nodo padre, si está explotado o no).

Una estructura superior al nodo la constituye la lista de nodos. Dicha lista almacenará todos los nodos abiertos (podría almacenar todos los nodos).

Mientras haya algún nodo abierto el proceso se repetirá con las siguientes 3 fases: seleccionar el siguiente nodo a explotar, explotar el nodo seleccionado (que incluye evaluar los nuevos nodos y cerrar el nodo ya explotado) y una última fase que cierra aquellos nodos que no se tengan, o no se quieran, mantener abiertos. Además cada vez que se genere una solución completa hay que comprobar si mejora la mejor solución disponible hasta este momento

```
GenerarNodoOriginal;  
GenerarUnaSolucionInicial; // esto es opcional  
While haya_nodos_abiertos do begin  
  SeleccionarNodoExplotar;  
  ExplotarNodoSeleccionado; // incluye EvaluarNuevosNodos;  
  ComprobarSiSeHaAlcanzadoUnaSoluciónMejor;  
  CerrarNodosNoÚtiles;  
End; //del while
```

A continuación se desarrollan las funciones básicas que hacen falta para el uso de esta estructura que son:

- a) La función de evaluación de cada nodo.
- b) El modo de seleccionar el nodo a explotar.
- c) El modo de explotar el nodo seleccionado.
- d) Los criterios por los cuales se cierran nodos

3.3.3 Funciones de evaluación

La función de evaluación considera dos paquetes de información que suministra la solución parcial que, como tal, tiene el nodo. Por una lado la parte de la solución que ya está completamente detallada. Por el otro la información asociada a la parte de la solución que no se ha definido.

Según lo dicho la función de evaluación del nodo, sea $f(n)$, se compone de una parte ligada la solución parcial conocida, sea $g(n)$ y sea $h(n)$ la evaluación de la solución que quedaría por definir.

Si el componente de la función de evaluación del nodo ligado a la fase ya definida de la solución se conoce de modo exacto se definirá como $g^*(n)$. Si el componente de la función de evaluación del nodo ligado a la parte no definida de la solución se conoce de modo exacto se definirá como $h^*(n)$.

Teniendo en cuenta la existencia de estos dos componentes se pueden definir diferentes modos de evaluar la calidad de cada nodo (que servirá posteriormente para compararlo entre ellos). Algunos de los parámetros que permiten crear los diferentes criterios son:

De modo general se puede definir $f(n)=(2-w(n))g(n)+w(n)h(n)$, donde $w(n)$ es un coeficiente que pondera la importancia de g y h en función del nodo en el que se encuentre la exploración.

En general, y salvo que se tenga una razón fundada, se puede asumir que $w(n)=1$ para todo n .

Es importante destacar que, en ocasiones, $g(n)$ y $h(n)$ serán muy costosos de calcular. Este hecho requiere una evaluación especial dado que el número de ocasiones en que haya que hacer el cálculo puede ser considerable.

3.3.4 Selección del Nodo a explotar

De entre los nodos abiertos hay que elegir cual es el nodo a explotar. Cada criterio tiene sus ventajas e inconvenientes ligadas a:

- b) La facilidad de búsqueda del nodo.
- c) La rapidez en la que se encuentran soluciones completas.
- d) La rapidez en la que se encuentran buenas soluciones completas.

Los criterios más habituales son:

- a) Selección aleatoria
- b) Primero el más antiguo (FIFO)
- c) Primero el más moderno (LIFO)
- d) Primero el más profundo (búsqueda en profundidad)
- e) Primero el menos profundo (Búsqueda en Anchura)
- f) Primero el de menor coste ($g(n)$)
- g) Primero el de mejor cota ($f(n)$),
- h) Primero el de mejor esperanza

3.3.5 Generación de nodos (explosión)

Cada problema tiene un mecanismo de generación de nodos (una vez seleccionado el mismo) diferente. No es posible proponer un mecanismo general. Aunque sí hay algunos criterios que se deben observar. En primer lugar debe generar todas las opciones posibles a partir del nodo explotado, y no debe conducir a la generación de una cantidad excesiva de nodos en cada explosión.

En los problemas en que la solución es una permutación de valores un modo habitual de explotar nodos es añadir un nuevo valor a la solución parcialmente construida.

3.3.6 Eliminación (o cierre) de Nodos.

No todos los nodos abiertos son igualmente necesarios, por ello se suele añadir un procedimiento que elimine aquellos nodos que no se quieran mantener abiertos.

Algunos de los nodos abiertos serán manifiestamente incompatibles y por ello deben ser cerrados. Otros nodos serán estructuralmente iguales que otros nodos ya creados (abiertos o no), podría interesar buscar esos nodos para cerrar el nodo idéntico.

Es posible que haya nodos que conduzcan de manera unívoca a la generación de una solución completa. En ese caso hay que crearla y valorarla, comparándola con la mejor solución disponible hasta el momento, guardándola si es mejor que ella y en cualquier caso cerrándolas.

Algunos nodos estarán dominados por otros nodos ya creados, en ese caso se debe cerrar el nodo. Otros nodos podrían tener alguna característica que les impidiera conducir a un óptimo en cuyo caso pueden también ser cerrados.

En algunos casos la cota de los nodos abiertos es ya peor que la mejor de las soluciones obtenidas. En ese caso no tiene sentido seguir explotando ese nodo y conviene que sea cerrado.

Por último, es posible que por problemas de memoria no se pueda tener abierta más de una cierta cantidad de nodos. En ese caso hay que elegir el conjunto de nodos a mantener y cerrar el resto.

Los procedimientos de enumeración completa analizan todos y cada uno de los nodos que se abren.

3.4 Otras técnicas de Enumeración basadas en la exploración por nodos.

Las técnicas de Enumeración Implícita analizan que nodos no se deben abrir puesto que se puede deducir de su estructura los resultados que puede obtener.

Las técnicas Beam Search avanzan por niveles y no permiten que haya más de M nodos abiertos simultáneamente.

Las técnicas Laser generan, a partir de cada nodo abierto, soluciones, mediante heurísticas rápidas, aunque la exploración se hace también por niveles. Este método permite tener buenas soluciones sin necesidad de acabar el procedimiento que garantizaría el óptimo.

4 PROCEDIMIENTOS DE MEJORA LOCAL

Son procedimientos basados en analizar el vecindario de una determinada solución, para averiguar si existe, en dicho vecindario, una solución con mejor valor de función objetivo.

El proceso de búsqueda examina todas las opciones dentro del vecindario actual y selecciona (generalmente) la mejor opción para moverse hacia él y recomenzar el proceso

4.1 Vecindario

La definición de vecindario es el conjunto de soluciones que pueden ser alcanzados desde la solución actual con una operación simple. Cualquier punto que requiera dos operaciones no es vecindario.

La mejor solución en un vecindario es un óptimo con respecto a su vecindario.

A continuación se describen los tres tipos de vecindario más habituales.

4.1.1 Intercambio de 2 elementos

Son vecinos de una determinada solución todas aquellas soluciones alcanzables mediante el intercambio de dos elementos de la solución.

Ejemplo: Sea ABCDEFGH la solución original considerada. Un intercambio de los elementos 3 y 6 proporcionaría la siguiente solución ABFDECGH

4.1.2 2-opt

Son vecinos de una determinada solución todas aquellas soluciones mediante el siguiente movimiento: Tras seleccionar dos elementos de una solución invertir el tramo que hay entre los mismos. Este tipo de vecindario ha demostrado su superioridad, en general, frente al del intercambio simple. Además todos los vecinos según el proceso de intercambio son vecinos en segundo nivel del vecindario 2-opt. Lo mismo ocurre con los vecinos encontrados mediante el proceso Insertar.

Ejemplo: Sea ABCDEFGH la solución original considerada. Un 2-opt de los elementos 3 y 6 proporcionaría la siguiente solución ABFEDCGH

4.1.3 Insertar

Son vecinos de una solución mediante el proceso de inserción aquellos en los que se coge un elemento y se inserta entre otros dos elementos. Los vecinos así conseguidos son vecinos en segundo nivel del vecindario 2-opt.

Ejemplo: Sea ABCDEFGH la solución original considerada. Insertar 3 en 6 proporcionaría la siguiente solución ABDEF CGH.

4.2 Algoritmos de Mejora basados en Vecindario

4.2.1 Nomenclatura

x: solución actual
x': solución en el vecindario
c(x) = valor de la función objetivo para x
N(x): vecindario de x
random: función

4.2.2 Mejora Iterativa Simple. Procedimiento

Paso 1) Inicialización
1.1 Seleccionar una solución inicial x
1.2 Definir mejor_coste:=c(x)
Paso 2) Selección (y acabar si no hay mejora)
2.1 Seleccionar x' perteneciente N(x) tal que $c(x') < c(x)$
2.2 Si no hay tal x' entonces STOP
Paso 3) Actualizar
3.1 $x := x'$
3.2 mejor_coste:=c(x)
3.3 Ir a paso 2.

4.2.3 Descenso Rápido. Procedimiento

Paso 1) Inicialización
1.1 Seleccionar una solución inicial x
1.2 Definir mejor_coste:=c(x)
Paso 2) Selección (y acabar si no hay mejora)
2.1 Seleccionar x' perteneciente N(x) tal que $c(x') < c(x)$ y es el menor coste en el vecindario $c(x') = \min(N(x))$
2.2 Si no hay tal x' entonces STOP
Paso 3) Actualizar
3.1 $x := x'$
3.2 mejor_coste:=c(x)
3.3 Ir a paso 2.

4.2.4 Mejora Iterativa Aleatorizada. Procedimiento

wp: real positivo menor que 1.
Paso 1) Inicialización
1.1 Seleccionar una solución inicial x
1.2 Definir mejor_coste:=c(x)
Paso 2) Selección
2.1 Si $\text{random} < \text{wp}$ seleccionar x' perteneciente N(x) de modo aleatorio
2.2 en caso contrario Seleccionar x' perteneciente N(x) tal que $c(x') < c(x)$
Paso 3) Actualizar
3.1 $x := x'$
3.2 mejor_coste:=c(x)
3.3 Mientras no se produzca la condición de terminación ir a paso 2.

5 PROCEDIMIENTOS METAHEURÍSTICOS

Los procedimientos metaheurísticos son un marco de referencia de alto nivel que se especializa para resolver problemas de optimización. Es también una estrategia que guía otras heurísticas en la búsqueda de mejores soluciones factibles.

Las metaheurísticas deberían ser generales y simples. A medida que se especializan pierden la generalidad y exigen más conocimiento del problema específico,

Además de los parámetros que definen cada algoritmo específico, las metaheurísticas requieren algún tipo de representación que por tanto es dependiente de cada problema.

Entre las metaheurísticas más conocidas se puede destacar:

- Algoritmos Genéticos
- Scatter Search
- Algoritmos Meméticos
- Algoritmos de Hormigas
- Redes Neuronales
- GRASP.
- Recocido Simulado
- Búsqueda Tabú
- Iterated Local Search, Variable Neighborhood Search

5.1 Procedimientos de población

Según (Osman, 1995) las metaheurísticas de población, a la que pertenecen los algoritmos genéticos, presentan todo o parte del siguiente esquema de la Ilustración 1: Estructura genérica de las Metaheurísticas (Osman, 1995) En este esquema se puede observar que todos los procedimientos empiezan obteniendo una generación inicial (que algunas veces pueden estar constituidas por un solo individuo). Esta generación pasa a una segunda etapa (búsqueda por generación de movimientos) o a una tercera (búsqueda por generación de soluciones). En la segunda etapa puede ser usado cualquier procedimiento de búsqueda local que generen vecindarios a partir de una solución única. Los métodos de búsqueda por individuos pueden operar sobre cada individuo de la generación actual.

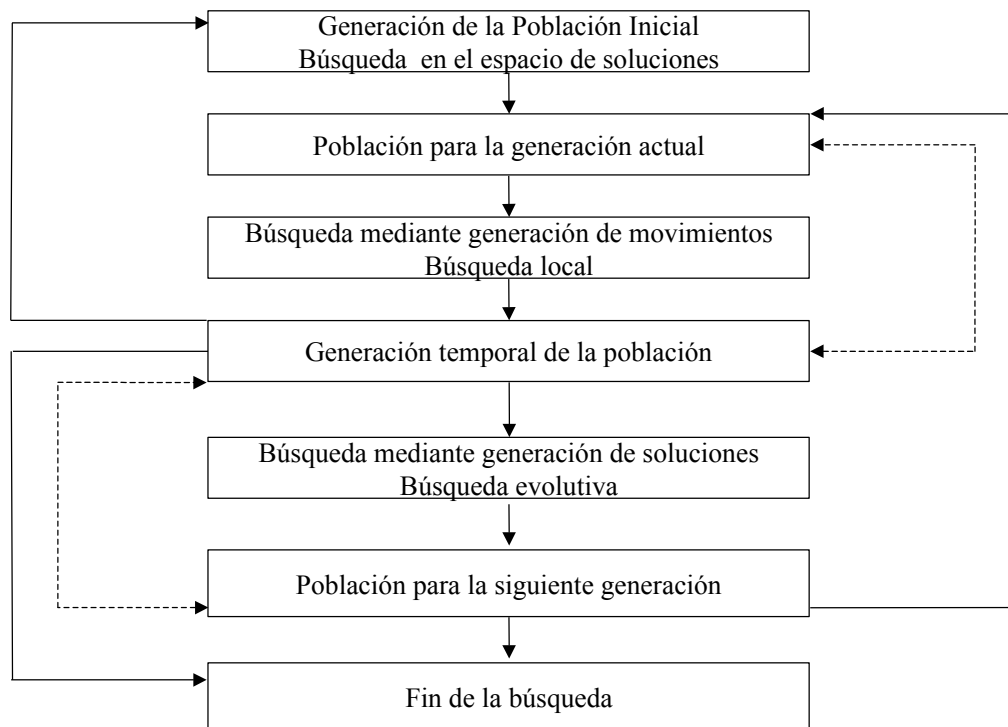


Ilustración 1: Estructura genérica de las Metaheurísticas (Osman, 1995)

Al final de la segunda etapa puede finalizar la búsqueda en la población actual e ir a la etapa cuatro o reinicializada volviendo a la etapa primera si la búsqueda evolutiva no se incluye, o la población actual pasa por la etapa tercera. En esta tercera etapa se realiza una búsqueda basada en la generación de nuevas soluciones. Al final de esta etapa se puede volver a la segunda o repetir otro ciclo de búsqueda evolutiva. Las líneas punteadas indican opciones que pueden ser utilizadas o no.

5.1.1 Algoritmos Genéticos

Los Algoritmos Genéticos fueron introducidos por Holland en 1975 para imitar algunos de los mecanismos que se observan en la evolución de las especies. Basándose en las principales características de la reproducción sexual, Holland creó un algoritmo que genera nuevas soluciones a partir de la unión de soluciones progenitoras utilizando operadores similares a los de la reproducción, sin necesidad de conocer el tipo de problema a resolver.

Una ventaja importante que presentan las heurísticas frente a las técnicas que buscan soluciones exactas es que, por lo general, permiten una mayor flexibilidad para el manejo de las características del problema. No suele ser complejo utilizar algoritmos heurísticos que en lugar de funciones lineales utilicen no linealidades. Habitualmente las heurísticas proponen un conjunto de soluciones, ampliando de esta forma las posibilidades de elección del decisor, especialmente cuando existen factores no cuantificables que no han podido ser reflejados en el modelo pero deben ser tenidos en cuenta.

En resumen, podría decirse que el uso de estas técnicas supone la posibilidad de resolver, de forma práctica, problemas de gran complejidad que resultaban intratables mediante técnicas exactas y permite definir con detalle el metabolismo del sistema.

Los algoritmos genéticos son una clase de algoritmos inspirados en los mecanismos de la genética, que se aplican a problemas de optimización (especialmente a los problemas de combinatoria).

“Procedimiento basado en la analogía con la evolución de los seres vivos. La premisa que subyace a este tipo de enfoques es, que se puede encontrar soluciones aproximadas a problemas de gran complejidad computacional mediante un procedimiento de evolución simulada matemáticamente en un ordenador “ [Holland, 1975]

Es una Metaheurística que permite a partir de una población inicial de soluciones, obtener soluciones potencialmente mejores mediante el cruce de varias de las soluciones originales.

Requieren el diseño de tres operadores (generalmente de tipo probabilístico) que actúan sobre objetos denominados “strings”.

- Reproducción: Selección de “strings” para poder proceder a la reproducción.
- Cruce: Combinación de dos o más strings para que intercambien valores, reproduciéndose..
- Mutación: Alteración espontánea de los elementos de un “string”.

Estos procesos pueden tener cada uno formas variadas, e incluso se permite el avance en paralelo de Algoritmos Genéticos con el mismo propósito.

5.1.1.1 Estructura General de un Algoritmo Genético.

Generación de la Población Inicial

Repetir

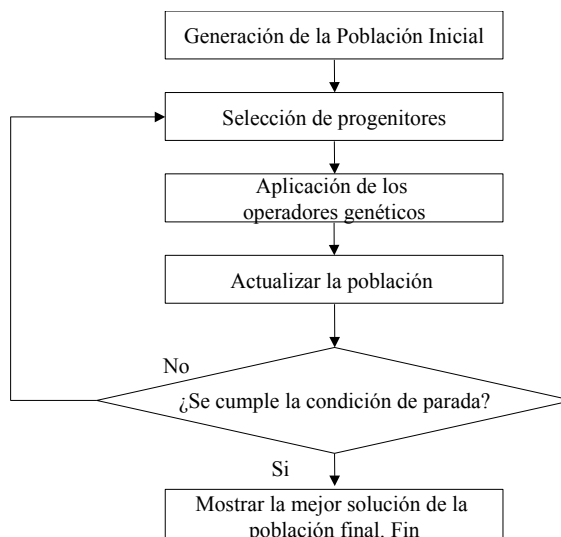
Elegir dos (o más) padres de entre la población.

Construir una solución nueva a partir de la estructura de los padres elegidos.

Si se considera conveniente provocar mutaciones.

Decidir si la nueva solución pasa a formar parte de la población.

Hasta alcanzar un criterio de parada



5.1.1.2 Representación de las soluciones (representación cromosómica).

Inicialmente en los Algoritmos Genéticos se representaba la solución mediante vectores de carácter binario. Los operadores eran muy sencillos pero muchas de las soluciones obtenidas eran infactibles que posteriormente había que reparar.

Otro modo de representar soluciones útil para los algoritmos genéticos es utilizar un vector unidimensional de elementos independientes. (Por ejemplo un vector para el Problema TSP).

Cuando el problema es más complejo (por ejemplo un problema de Flow Shop Híbrido) puede exigir N Vectores con codificación combinada. En el caso del FlowShop Híbrido un buen modo de representar es un vector que representa la secuencia de operaciones y otro vector las máquinas donde cada producto va destinado.

También se pueden utilizar vectores n-dimensionales cuyo problema fundamental es el de definir métodos de cruce eficientes (Ej: Vector Bidimensional para la Programación de Producción en Máquinas en Paralelo)

Por último existen vectores con codificación implícita como por ejemplo para la codificación del balance de líneas en los que una correcta codificación es una exigencia para obtener resultados razonables.

5.1.1.3 Definición de la Medida de Evaluación de cada Solución (Fitness)

El procedimiento de evaluación de una solución es especialmente importante en un algoritmo genético, pues se va a utilizar muchas veces.

- $Fitness(x)$ = Valor función objetivo
- $Fitness(x)$ = Valor función objetivo normalizada
- $Fitness(x)$ = Valor función objetivo normalizada –Penalización por infactibilidad
- Otros ...

5.1.1.4 Definición de Métodos para la Generación de la Población inicial

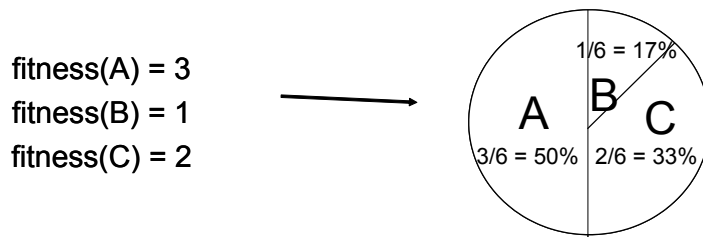
Procedimientos

- Generación Aleatoria
- Heurísticas conocidas

El tamaño de la Población es especialmente relevante. Un tamaño demasiado grande genera dispersión de resultados Un tamaño demasiado pequeño genera una excesivamente rápida convergencia de soluciones. Una pregunta interesante al diseñar un algoritmo genético es si debe mantenerse constante a lo largo del proceso.

5.1.1.5 Definición de Métodos para la selección de dos (o más) padres de la Población Inicial.

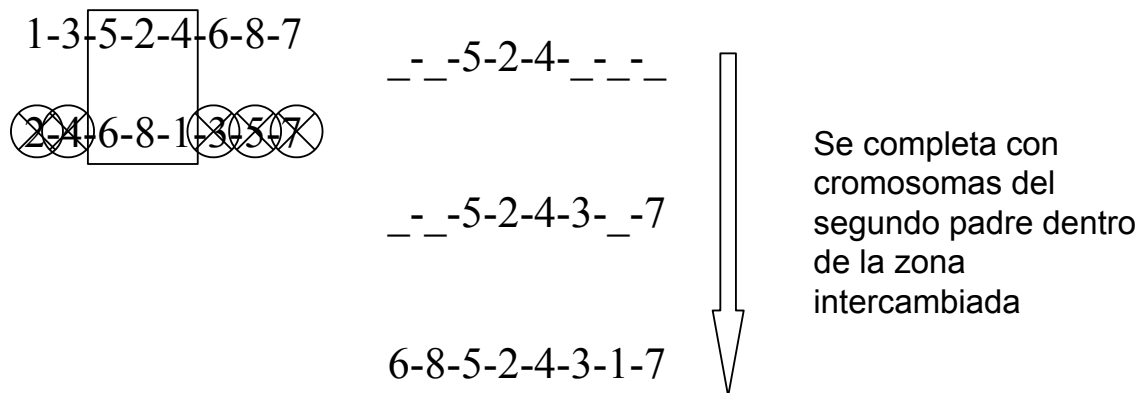
El número de padres seleccionados es generalmente dos (aunque puede ser mayor si el procedimiento de cruce lo permite). El modo de selección de los padres se suele realizar mediante el uso de la denominada ruleta. Se asignan probabilidades a los padres en función de su fitness



5.1.1.6 Operaciones de Recombinación o Cruce de los individuos seleccionados.

El cruce o intercambio de características entre las soluciones elegidas produce un fenómeno de convergencia puesto que hace que los individuos de la población cada vez se parezca mas entre sí.

Existen infinidad de operadores de cruce, los más sencillos son los desarrollados para representaciones binarias como el one-point crossover que intercambia a partir de una posición determinada la cadena binaria que representa el individuo de cada padre. Este concepto se ha refinado con el two-point crossover que intercambia la parte del individuo que hay entre dos posiciones definidas de manera aleatoria.



5.1.1.7 Generación Soluciones Iniciales

Para generar las soluciones iniciales se puede optar por generarlos de manera aleatoria o utilizando heurísticas conocidas que generan buenas soluciones de partida.

El problema de la primera aproximación es que la convergencia a una buena solución puede ser lenta. El problema de la segunda aproximación es que tiene una tendencia inicial que será difícilmente superada.

Por otro lado el tamaño de la Población es especialmente relevante. Un tamaño demasiado grande genera dispersión de resultados Un tamaño demasiado pequeño genera una excesivamente rápida convergencia de soluciones. Otra cuestión a resolver es si debe mantenerse constante dicho tamaño a lo largo del Proceso.

5.1.1.8 Selección de “Padres”

El número de padres seleccionados es generalmente dos (aunque puede ser mayor si el procedimiento de cruce lo permite).

El modo de selección de los padres se suele realizar mediante el uso de la denominada ruleta. En ella se asigna una probabilidad a cada padre proporcional a su *fitness* de tal modo que la suma de probabilidades sea la unidad.

5.1.1.9 Cruce

El cruce o intercambio de características entre las soluciones elegidas produce un fenómeno de convergencia puesto que hace que los individuos de la población cada vez se parezca mas entre sí.

Existen infinidad de operadores de cruce, los más sencillos son los desarrollados para representaciones binarias como el one-point crossover que intercambia a partir de una posición determinada la cadena binaria que representa el individuo de cada padre. Este concepto se ha refinado con el “cruce de dos puntos” que intercambia la parte del individuo que hay entre dos posiciones definidas de manera aleatoria.

Este operador En primer lugar se calculan dos números aleatorios entre 1 y el número de elementos del vector. El operador obtiene del vector “progenitor a” los elementos contenidos entre el primer número y el segundo número. Posición a posición de entre las aún no asignadas se copian los elementos del “progenitor b” si no están incluidos en el fragmento de vector copiado del padre. Los huecos que queden se rellenan con el orden en el que aparecen los valores no incorporados del “progenitor b”.

En muchas ocasiones el resultado así obtenido no es una solución factible. En estos casos hay que proceder a realizar una operación de “reparación” en la que se genera factibilidad al resultado obtenido.

5.1.1.10 Actualizar Población

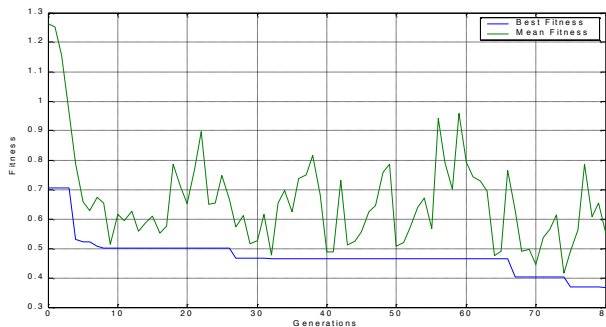
El argumento fundamental de los algoritmos genéticos no es que obtienen buenos resultados a partir de la exploración ligada a una población inicial, sino que la población evoluciona a medida que se obtienen soluciones mejores.

Dicha evolución se consigue por la vía de sustituir (de uno en uno o en paquetes) elementos de la población anterior por soluciones obtenidas durante la ejecución del algoritmo.

5.1.1.11 Monitorización de Convergencia y Mutación

Cada cierto tiempo es interesante realizar operaciones de mutación, para garantizar que la población no converja demasiado rápidamente. Generalmente son operaciones en las que el vector obtenido es afectado por alguna operación de búsqueda de vecindario como la inserción o el intercambio de dos elementos.

La monitorización de la convergencia permite lanzar procedimientos de mutación si es necesario. Un modo de monitorizar la convergencia es contabilizar para toda la población si se repiten los mismos valores en las posiciones.



Puede ser interesante aplicar cada cierto tiempo pequeñas mutaciones a las soluciones obtenidas, ya sea para mejorar la población, ya sea para evitar la excesiva convergencia en la población. Los tipos básicos de mutación recuerdan a los vecindarios definidos en otro lugar de estos apuntes (2-opt, Inversión, Inserción) pero también podría ser la introducción selectiva de elementos “ajenos” a la evolución de la solución.

5.1.2 Scatter Search y Algoritmos Meméticos

La Scatter Search o búsqueda dispersa es un método metaheurístico de población para resolver problemas de optimización. Aunque fue originalmente introducido en los años setenta, recientemente es cuando ha sido probado en numerosos problemas difíciles con gran éxito (Glover, Laguna, and Martí, 2003).

Pertenece a la familia de los llamados Algoritmos Evolutivos, y aunque presenta similitudes con los Algoritmos Genéticos, difiere de éstos en principios fundamentales, tales como el uso de estrategias sistemáticas en lugar de aleatorias o la posibilidad de combinar más de dos elementos.

Scatter Search proporciona un marco flexible que permite el desarrollo de diferentes implementaciones con distintos grados de complejidad. El método consta de las siguientes etapas: Generación de la Diversidad, Método de Mejora sobre la población anterior, selección de la población de referencia, selección de un subconjunto para la combinación, combinación y actualización de la población de referencia.

Comienza con una colección de puntos de referencia (obtenidos mediante la aplicación de otros procedimientos). Se parece a los algoritmos genéticos excepto que en la búsqueda “scatter” la combinación de elementos de la población es una combinación lineal que es posteriormente adaptada para convertirse en una solución.

Los algoritmos meméticos también pertenecen a los algoritmos basados en poblaciones. Explicado de una manera muy básica utiliza el operador de cruce de los algoritmos genéticos, pero al obtener un “hijo” le aplica métodos de mejora local.

5.2 Metaheurísticas de Búsqueda de Entornos

5.2.1 GRASP

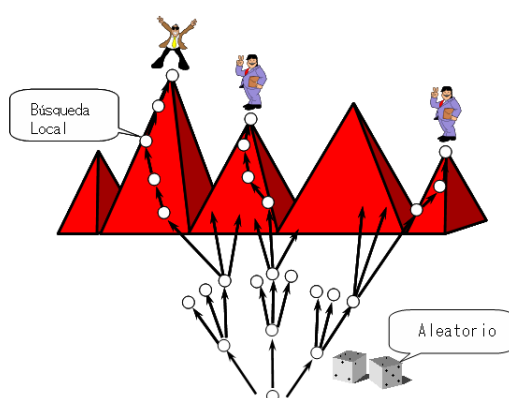
Probablemente el modo más sencillo para evitar los óptimos locales es empezar por puntos diferentes cada vez el camino de descenso (Marti, 2003). Esta es la lógica básica del procedimiento denominado Greedy Randomized Adaptive Search Procedure-GRASP- (Resende and Ribeiro, 2003). La generación de los puntos de inicio es la parte voraz, aleatoria y adaptativa del proceso mientras que la búsqueda local es el componente search. Este algoritmo de muy fácil implementación, puede ser también paralelizado o integrado con otras metaheurísticas.

GRASP es el acrónimo de Greedy Randomized Adaptive Search Procedures. Es una metaheurística que empieza la resolución del problema en diferentes puntos del espacio de soluciones y una vez “reempieza” utiliza un procedimiento heurístico para completar o mejorar la solución, alcanzando un óptimo local.

La definición de la situación inicial puede ser más o menos aleatoria. El método de resolución puede ser una heurística específica del problema u otra metaheurística de búsqueda local.

El modo más sencillo para evitar los óptimos locales es empezar por puntos diferentes cada vez el camino de descenso (Marti, 2003). Esta es la lógica básica del procedimiento denominado Greedy Randomized Adaptive Search Procedure-GRASP- (Resende *et al.*, 2003). La generación de los puntos de inicio es la parte voraz, aleatoria y adaptativa del proceso mientras que la búsqueda local es el componente search. Este algoritmo de muy fácil implementación, puede ser también paralelizado o integrado con otras metaheurísticas.

Otro modo de concebir las estructuras GRASP es mediante el uso de heurísticas constructivas randomizadas..



Dicha estructura utilizaría un algoritmo heurístico constructivo voraz, sustituyendo el criterio de selección del siguiente elemento a incorporar a la selección por un criterio estocástico. De este modo, en el problema TSP, en lugar de elegir, por ejemplo, el vecino más cercano, se elige aleatoriamente de entre los 3 vecinos más cercanos, o de entre aquellos vecinos que sólo estén un 10% más lejos que el más cercano de los vecinos.

5.2.2 ILS y VNS

La idea de reiniciar la búsqueda de nuevo es interesante, pero para espacios de solución extenso puede llevar a un procedimiento no eficiente. Por ello la Iterated Local Search (Lourenço *et al.*, 2003) lo que pretende es comenzar la búsqueda de nuevo no “por cualquier lado” si no por puntos del espacio que son modificaciones –no demasiado fuertes- de soluciones ya encontradas previamente. Específicamente se puede citar la Búsqueda de Vecindario Variable (Variable Neighborhood Search-VNS), cuya idea fundamental es la búsqueda de mejores soluciones cambiando de vecindario cuando el que se está considerando no va a dar mejores resultados (Hansen *et al.*, 2003). La VNS se comporta como un método de descenso hacia mínimos locales, explorando sistemáticamente o aleatoriamente, cada vez más lejanos vecindarios de la solución. El método salta de la solución actual a una mejor si y sólo si encuentra una mejor solución. (Perez *et al.*, 2006).

5.2.3 Recocido Simulado

Los algoritmos de Recocido Simulado (Simulated Annealing) fueron introducidos por Cerny y Kirkpatrick a principio de la década de los 80 del siglo XX para la optimización de problemas combinatorios con mínimos locales. Utilizan técnicas de optimización no determinista: no buscan la mejor solución en el entorno de la solución actual sino que generan aleatoriamente una solución cercana y la aceptan como la mejor si tiene menor coste, o en caso contrario con una cierta probabilidad p ; esta probabilidad de aceptación irá disminuyendo con el número de iteraciones y está relacionada con el empeoramiento del coste. (Henderson *et al.*, 2003)

El recocido simulado está basado en un cómo funciona el proceso físico de recocido. En el proceso de búsqueda de soluciones permite movimientos que no mejoran con una probabilidad que decrece a lo largo del tiempo. El ratio de dicho decrecimiento queda determinado por el programa de enfriamiento que casi siempre es un parámetro usado como ratio de decrecimiento exponencial.

```
x: solución actual
c(x) = valor de la función objetivo para x
N(x): vecindario de x

Paso 1) Inicialización
  1.1 Seleccionar una solución inicial x
  1.2 Definir mejor_coste:=c(x)
  1.3 Determinar la T inicial
Paso 2) Mientras no se produzca la condición de terminación
  2.1 Elegir un x' perteneciente N(x)
  2.2 Si x' es aceptable dependiendo del criterio de Recocido Simulado.
      x:=x'
  2.3 Actualizar T de acuerdo a la planificación de Recocido Simulado.
```

5.2.4 Tabu search.

Otro método metaheurístico que pretende dotar de "inteligencia" a los algoritmos de búsqueda local es denominado de Búsqueda Tabú (Gendreau, 2003). La búsqueda tabú, a diferencia de otros algoritmos basados en técnicas aleatorias de búsqueda en vecindario, se caracteriza porque utiliza una estrategia basada en el uso de estructuras de memoria para escapar de los óptimos locales, en los que

se puede caer al "moverse" de una solución a otra por el espacio de soluciones. Al igual que en la búsqueda local, la búsqueda tabú selecciona de modo agresivo el mejor de los movimientos posibles en cada paso. Al contrario que sucede en la búsqueda local, se permiten movimientos a soluciones del entorno aunque se produzca un empeoramiento de la función objetivo, de manera que sea posible escapar de los óptimos locales y continuar estratégicamente la búsqueda de mejores soluciones.

La búsqueda tabú es una metaheurística que resuelve problemas de optimización, basándose en que se prohíbe al sistema deshacer algunos de los últimos movimientos realizados, para permitir la búsqueda por entornos que de otro modo no se explorarían.

Esta metaheurística resuelve problemas de optimización, basándose en una gestión de memoria multinivel y exploración. Utiliza el concepto de vecindario para probar soluciones. En su versión más simple un procedimiento de búsqueda tabú es como sigue:

```
x: solución actual
c(x) = valor de la función objetivo para x
N(x): vecindario de x

x:=determinarSoluciónInicial;
while not(hayQueTerminar) do begin
  V:=conjuntoDeVecinosNoProhibidos(x);
  x':=elegirMejorVecino(V);
  actualizarListaDeMovimientosProhibidos(x,x');
  x:=x';
end;
```

Hay muchas variaciones, como por ejemplo los niveles de aspiración, que se pueden incluir en especificaciones más complejas.

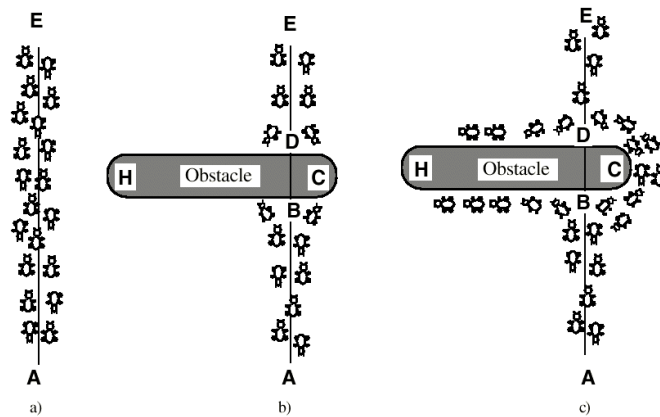
5.3 Metaheurísticas basadas en el reconocimiento de patrones.

5.3.1 Redes Neuronales

Las redes neuronales artificiales, son sistemas de procesamiento de la información cuya estructura y funcionamiento emulan a las redes neuronales biológicas. Los elementos simples de procesamiento de una red neuronal artificial también se denominan neuronas y el peso de sus conexiones, pesos sinápticos. Las redes neuronales tienen una gran utilidad en sistemas de predicción. Mediante un histórico de datos (y de salidas) se establecen las relaciones entre los datos y las salidas (entrenamiento), de tal modo que a un nuevo juego de datos le correspondería una nueva salida.

5.3.2 Algoritmo de Colonia de Hormigas

Un procedimiento heurístico de búsqueda basado en el comportamiento de las colonias de hormigas. Particularmente su habilidad para encontrar el camino más corto de modo colectivo.



Las hormigas tienen memoria que afecta al proceso de construcción del camino de modo que aquel camino que es de menor camino es el que tiene más probabilidad de ser elegido. En los algoritmos ACO se guarda memoria de las conexiones más habituales entre dos elementos consecutivos.

6 ALGUNOS EJERCICIOS DE OPTIMIZACIÓN COMBINATORIA

6.1 Problema del FlowShop de 3 máquinas.

6.1.1 Descripción del Problema

Sea un taller de flujo compuesto por M máquinas donde se deben secuenciar N productos, que para su elaboración pasarán de manera consecutiva por cada una de las M máquinas, con unos tiempos de ejecución de cada producto i en cada máquina de $p(i,j)$ respectivamente.

Se trata de definir la secuencia que minimice el tiempo de flujo máximo (C_{max}) que se define como el lapso de tiempo mínimo que transcurrirá desde que el primer producto se empieza a producir en la primera máquina hasta que el último se acaba de producir en la última máquina.



6.1.2 Definición de la estructura de la solución.

Una solución es la permutación que indica el orden en que se deben ejecutar los trabajos.

Una solución se puede representar por un vector $v[i]$ que define de modo ordenado la secuencia de trabajo

6.1.3 Definición del modo de evaluar la solución.

Dada una solución $v(i)$, se trataría de calcular cuando acaba de producirse el producto en posición n, es decir $v(n)$

```
Maq[1]:=0;Maq[2]:=0;Maq[3]:=0;
for i=1 to n do begin
  Maq[1]:=Maq[1]+p[v[i],1];
  Maq[2]:=Max(Maq[1],Maq[2])+p[v[i],2];
  Maq[3]:=Max(Maq[2],Maq[3])+p[v[i],3];
end;
Cmax=Maq[3];
```

6.1.4 Un procedimiento de generación aleatoria de soluciones.

```
Sea v un vector con los n elementos a permutar
i:=0;
While i<n-1 do begin
  j:=random(1..n-i)
  intercambiar(v[i+1];v[i+j])
  i:=i+1
end; // del while
```

6.1.5 Un procedimiento enumerativo de resolución.

Un procedimiento enumerativo basado en ramificación y poda requiere definir la función de evaluación de cada nodo, definir el modo en que se va a seleccionar el nodo a explotar, definir el modo de explosión, definir el primero de los nodos y definir el modo de eliminar los nodos.

```
generarElNodoInicial;
generarUnaSoluciónInicial;
Mientras haya nodos abiertos;
  Elegir un nodo (por ejemplo según menor f(n))
  Explotar el nodo (añadir un producto de los no secuenciados a la secuencia).
  Evaluar f(n) para las nuevos nodos.
  Eliminar los nodos para los cuales el valor de f(n) es mayor que la mejor solución ya obtenida.
  Si el número de elementos a añadir es 0, evaluar el nodo y comparar contra la mejor solución obtenida.
  Guardar en su caso.
```

- Definir $f(n)$
 - $f(n)=g(n)+h(n)$
 - $g(n)$ =el momento en el que acaba el último producto de la secuencia ya generada en la última máquina
 - $h(n)$ =la suma de las operaciones en la última máquina de los productos no secuenciados.
- Definir el modo de selección del nodo a explotar
 - Por ejemplo el de menor $f(n)$
- Definir el modo de explotar nodos
 - Añadir un producto aún no secuenciado a la secuencia.
- El primer nodo es la secuencia vacía.

6.1.6 Un procedimiento heurístico

Se puede establecer un procedimiento heurístico por la vía de generar una lista ordenada según un determinado criterio y secuenciar según dicho criterio.

```
Sea v un vector con los n elementos a permutar
Calcular para cada producto  $prod[i] = p[i,1] + p[i,2] + p[i,3]$ ;
Ordenar los productos según prod crecientes
Asignar a v la anterior ordenación.
```

```
Sea v un vector con los n elementos a permutar
Calcular para cada producto  $prod(i) = \sum_{j=1..3} (2j-1) * p(i,j)$ 
Ordenar los productos según prod decrecientes
Asignar a v la anterior ordenación.
```

6.1.7 Un procedimiento de mejora local

Definir un procedimiento de mejora local exige: Seleccionar una definición de vecindario, Seleccionar un proceso de búsqueda local y seleccionar el modo de generación de una solución inicial.

El vecindario seleccionado es el de intercambiar dos elementos de la permutación.

El proceso de mejora local seleccionado es Mejora Iterativa Simple.

La solución inicial se genera aleatoriamente.

```
generarSolucionAleatoria;
coste:=evaluarSolución(v);
i:=1;
while (i<=n-1) do begin
  j:=i;
  HayMejora:=false;
  while ((j<=n) and not(HayMejora)) do begin
    j:=j+1;
    intercambiar(v[i],v[j]);
    if evaluarSolucion(v)<=coste then HayMejora:=true else deshacerIntercambio(v[i],v[j]);
  end;
  if j=n then i:=i+1;
  if HayMejora then i:=1;
end; // del while
```

6.1.8 Un algoritmo genético

La estructura de un algoritmo genético es:

```
generarPoblacionInicial;
while not(hayQueParar) do begin
  seleccionarPadres;
  construirNuevaSolucion;
  provocarMutaciones;
  actualizarPoblación;
end;
```

Por tanto para definir el procedimiento hace falta:

Definir un tamaño de población inicial: 20

Definir un modo de generación de la población inicial: aleatoriamente

Definir un modo de parar: al cabo de 1000 iteraciones.

Se selecciona un modo de selección de padres en función de una ruleta donde el valor de fitness sea la inversa de la valoración de la solución.

Se define un procedimiento de cruce (por ejemplo OX con dos puntos de cruce).

Se actualiza la población: Si una solución obtenida del cruce es mejor que la peor de las soluciones la sustituye. Si no es mejor se calcula una mutación mediante un intercambio de dos elementos y se incorpora a la población eliminando una solución cualquiera elegida al azar.

6.2 Problema del viajante de comercio.

6.2.1 Descripción del Problema.

Sea un conjunto de puntos de un plano a una cierta distancia unos de otros.

Se trata de unir todos los puntos mediante un circuito que sólo visite cada nodo en una ocasión minimizando el recorrido

6.2.2 Definición de la estructura de la solución.

La solución se puede representar mediante una permutación de los puntos a visitar.

6.2.3 Definición del modo de evaluar la solución.

```
coste:=0
For i:=1 to n-1 do begin
  coste=coste+distancia[v[i],v[i+1]];
End
coste=coste+distancia[v[n],v[1]];
```

6.2.4 Un procedimiento de generación aleatoria de soluciones.

```
Sea v un vector con los n elementos a permutar
i:=0;
While i<n-1 do begin
  j:=random(1..n-i)
  intercambiar(v[i+1];v[i+j])
  i:=i+1
end; // del while
```

6.2.5 Un procedimiento enumerativo de resolución.

Un procedimiento enumerativo basado en ramificación y poda requiere definir la función de evaluación de cada nodo, definir el modo en que se va a seleccionar el nodo a explotar, definir el modo de explosión, definir el primero de los nodos y definir el modo de eliminar los nodos.

```
generarElNodoInicial;
generarUnaSoluciónInicial;
Mientras haya nodos abiertos;
  Elegir un nodo
  Explotar el nodo.
  Evaluar f(n) para los nuevos nodos.
  Eliminar los nodos para los cuales el valor de f(n) es mayor que la mejor solución ya obtenida.
  Si el número de elementos que quedan por añadir es 0, evaluar el nodo y comparar contra la mejor solución obtenida. Guardar en su caso.
```

- Definir $f(n)$
 - $f(n)=g(n)+h(n)$
 - $g(n)$ = es la suma de las distancias entre los puntos ya asignados

- $h(n)$ =el número de nodos por asignar multiplicado por el menor valor de la matriz distancia más el menor valor en la columna del primer elemento de la secuencia.
- Definir el modo de selección del nodo a explotar
 - Por ejemplo el de mayor profundidad(mayor número de ciudades incorporadas) en caso de empate utilizar el menor valor de $f(n)$
- Definir el modo de explotar nodos
 - Añadir todos los puntos aún no visitado a la secuencia.
- El primer nodo contiene un punto cualquier al azar al azar.

6.2.6 Un procedimiento heurístico

Un procedimiento heurístico de construcción sería el del vecino más próximo.

Sea v un vector con todos los nodos a visitar

```
i:=1
while i<=n-1 do begin
  seleccion:=i+1;
  for j:=i+2 to n do begin
    if distancia[v[i],v[j]]< distancia[v[i],v[seleccion]] then seleccion:=j;
  end; // del for
  intercambiar[v[i+1],v[seleccion]];
end; //del while
```

Este procedimiento podría ser mejorado si se repitiera N veces y en cada ocasión se hiciera empezar el ciclo por un nodo diferente y se guardara el mejor resultado. Además se podría comenzar el tour por el último nodo, aunque este procedimiento sólo produciría mejoras en caso de matrices asimétricas.

6.2.7 Un procedimiento de mejora local

Definir un procedimiento de mejora local exige: Seleccionar una definición de vecindario, Seleccionar un proceso de búsqueda local y seleccionar el modo de generación de una solución inicial.

El vecindario seleccionado es el de provocar un 2-opt (o mutación inversa).

El proceso de mejora local seleccionado es Mejora Iterativa Simple.

La solución inicial se genera mediante la HeurísticaVecino más Cercano.

```
generarSolucionVecinoMasCercano;
coste:=evaluarSolucion(v);
i:=1;
while (i<=n-1) do begin
  j:=i;
  HayMejora:=false;
  while ((j<=n) and not(HayMejora)) do begin
    j:=j+1;
    mutaciónInversa(i,j);
    if evaluarSolucion(v)<=coste then HayMejora:=true else mutaciónInversa(j,i);
  end;
  if j=n then i:=i+1;
  if HayMejora then i:=1;
end; // del while
```

6.2.8 Un algoritmo genético

La estructura de un algoritmo genético es:

```
generarPoblacionInicial;  
while not(hayQueParar) do begin  
  seleccionarPadres;  
  construirNuevaSolucion;  
  provocarMutaciones;  
  actualizarPoblación;  
end;
```

Definir un tamaño de población inicial: 20

Definir un modo de generación de la población inicial: aleatoriamente

Definir un modo de parar: al cabo de 1000 iteraciones.

Se selecciona un modo de selección de padres en función de una ruleta donde:
 $\text{fitness}(v) = 1/\text{Evaluación}(v)$

Se define un procedimiento de cruce (por ejemplo OX).

Se define un procedimiento de actualización: De cada 100 iteraciones se guardan las 18 mejores, dos soluciones se calculan aleatoriamente y se incorporan a la población.

Cada 20 iteraciones se elige al azar un elemento de la población y se le hace una mutación inversa de modo aleatorio.

```
generarPoblacionInicial;  
while not(hayQueParar) do begin  
  seleccionarPadres;  
  construirNuevaSolucion;  
  provocarMutaciones;  
  actualizarPoblación;  
end;
```

6.3 Problema de Secuenciación JIT

6.3.1 Descripción del Problema

Sea una línea de montaje donde se ensamblan 3 modelos de un determinado producto. De cada uno de los modelos hay que fabricar una cantidad $U[i]$ determinada. Se pretende, para garantizar la regularidad, que es básica en los sistemas Just-In-Time, que la producción salga de modo tan equilibrado como sea posible.

6.3.2 Definición de la estructura de la solución.

Una solución es una concatenación de los diferentes tipos de productos, de tal manera que la cantidad de cada tipo es al final igual a la demandada $U[i]$.

6.3.3 Definición del modo de evaluar la solución.

El criterio más habitual para medir la calidad de una solución es intentando evaluar la distancia que separa la producción acumulada en un determinado instante de la secuencia, de la que idealmente habría que haber fabricado.

$$\sum_i (Y_{i,k} - r_i k)^2$$

Donde $Y[i,k]$ es la producción acumulada hasta k del producto de tipo i .

$r[i]$ es el ratio de producción de i , $r[i]=U[i]/T$

T es el número total de unidades a producir, es decir la suma de $U[i]$ para todo i .

6.3.4 Un procedimiento de generación aleatoria de soluciones.

```
k:=0;
for i:=1 to P do begin
  for h:=1 to U[i] do begin
    k:=k+1;
    v[k]:=i;
  end;
end;
T:=k;
h:=0;
While h<T-1 do begin
  j:=random(1..T-h)
  intercambiar(v[h+1];v[h+j])
  h:=h+1
end; // del while
```

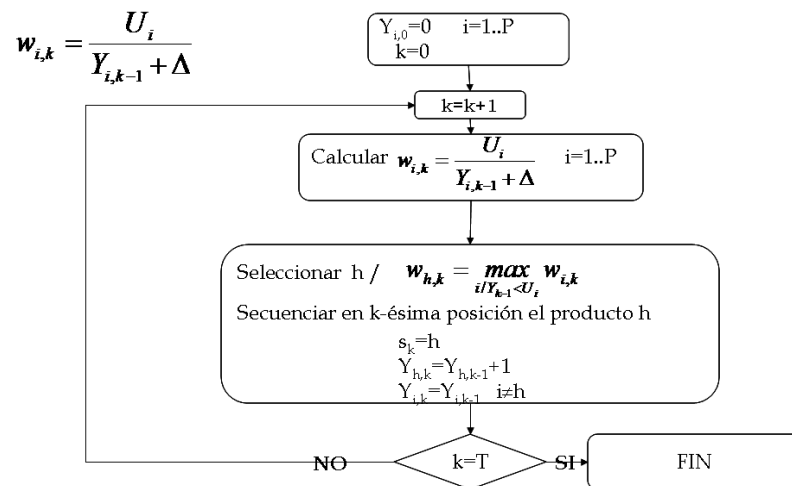
6.3.5 Un procedimiento enumerativo de resolución.

Un procedimiento enumerativo basado en ramificación y poda requiere definir la función de evaluación de cada nodo, definir el modo en que se va a seleccionar el nodo a explotar, definir el modo de explosión, definir el primero de los nodos y definir el modo de eliminar los nodos.

```
generarElNodoInicial;
generarUnaSoluciónInicial;
Mientras haya nodos abiertos;
  Elegir un nodo
  Explotar el nodo.
  Evaluar  $f(n)$  para los nuevos nodos.
  Eliminar los nodos.
  Si el número de elementos que quedan por añadir es 0, evaluar el nodo y comparar contra la mejor solución obtenida. Guardar en su caso.
```

- Definir $f(n)$
 - $f(n)=g(n)+h(n)$
 - $g(n)$ = es la valoración de la función hasta el punto de creación.
 - $h(n)=0$.
- Definir el modo de selección del nodo a explotar
 - Por ejemplo el de menor profundidad(menor número de unidades ya incorporados) en caso de empate utilizar el menor valor de $f(n)$
- Definir el modo de explotar nodos
 - Añadir un producto (todos los productos) de los que no se han fabricado toda la secuencia.
- Eliminar aquellos nodos donde la cantidad de productos ya secuenciados de cada tipo sea idéntico a otro con mejor valoración.
- El primer nodo está vacío.

6.3.6 Un procedimiento heurístico



La modificación del valor de Δ permitirá obtener diferentes secuencias con el mismo procedimiento.

6.3.7 Un procedimiento de mejora local.

Definir un procedimiento de mejora local exige: Seleccionar una definición de vecindario, Seleccionar un proceso de búsqueda local y seleccionar el modo de generación de una solución inicial.

El vecindario seleccionado es el de provocar una inserción de un elemento elegido al azar en una posición elegida al azar.

El proceso de mejora local seleccionado es Descenso Rápido.

La solución inicial se genera aleatoriamente.

```

generarSolucionVecinoMasCercano;
coste:=evaluarSolución(v);
i:=1;
while (i<=n-1) do begin
  j:=i;
  HayMejora:=false;
  while ((j<=n) and not(HayMejora)) do begin
    j:=j+1;
    mutaciónInversa(i,j);
    if evaluarSolucion(v)<=coste then HayMejora:=true else mutaciónInversa(j,i);
  end;
  if j=n then i:=i+1;
  if HayMejora then i:=1;
end; // del while
  
```

6.3.8 Un algoritmo genético

La estructura de un algoritmo genético es:

```

generarPoblacionInicial;
while not(hayQueParar) do begin
  seleccionarPadres;
  construirNuevaSolucion;
  provocarMutaciones;
  actualizarPoblación;
end;
  
```

Por tanto para definir el procedimiento hace falta:

Definir un tamaño de población inicial: 20

Definir un modo de generación de la población inicial: aleatoriamente

Definir un modo de parar: al cabo de 1000 iteraciones.

Se selecciona un modo de selección de padres en función de una ruleta donde el valor de fitness sea la inversa de la valoración de la solución.

Se define un procedimiento de cruce: en este caso se propone un OX con un punto de cruce, donde la sección del Padre y la Madre se incorporan íntegramente y se repara (aquellos productos que se producen de más frente a los que se producen de menos) aleatoriamente quitando algún producto de los que hay de más para poner alguno de los que hay de menos (ambos seleccionados aleatoriamente).

Se actualiza la población: Si una solución obtenida del cruce es mejor que la peor de las soluciones la sustituye. Si no es mejor se calcula una mutación mediante un intercambio de dos elementos y se incorpora a la población eliminando una solución cualquiera elegida al azar.

6.4 Corte de Piezas rectangulares.

En una empresa de fabricación de piezas metálicas, tienen una sierra automática capaz de hacer cualquier tipo de corte rectangular.

La citada empresa compra planchas rectangulares de dimensiones DX y DY.

La explosión de la lista de materiales de la empresa a partir de los pedidos en firme genera un conjunto de N piezas rectangulares de dimensiones (a,b).

Se trata de diseñar un programa de corte para cada plancha que minimice los restos de cada plancha.

- a) Construya el Modelo que define el problema de corte.

Indices

i, j : Recorre los cortes

Parametros

DX, DY : Dimensiones X e Y de la Placa Origen

a_i, b_i : Dimensiones x e y de cada corte i

Variables

x_i, y_i : Posicion (x,y) de la esquina superior izquierda de la placa i

r_i, s_i : Posicion (x,y) de la esquina inferior derecha de la placa i

$\alpha_{i,j} = \{0,1\}$ que indica si un producto i esta mas a la izquierda que otro j

$\beta_{i,j} = \{0,1\}$ que indica si un producto i esta mas arriba que otro j

$\gamma_i = \{0,1\}$ que indica si la orientacion del producto es la natural o la perpendicular

t = maximo de todos los valores r_i

w = maximo de todos los valores s_i

Modelo

$$[\text{MAX}] (DX - t)DY + (DY - w)DX - wt$$

Sujeto a

$$r_i = x_i + b_i\gamma_i + a_i(1 - \gamma_i) \quad \forall i$$

$$s_i = y_i + a_i\gamma_i + b_i(1 - \gamma_i) \quad \forall i$$

$$t \geq r_i \quad \forall i$$

$$w \geq s_i \quad \forall i$$

$$r_i \leq DX \quad \forall i$$

$$s_i \leq DY \quad \forall i$$

$$\alpha_{i,j} = 1 \rightarrow (x_i + a_i \leq x_j) \quad \forall i \neq j$$

$$\beta_{i,j} = 1 \rightarrow (y_i + b_i \leq y_j) \quad \forall i \neq j$$

$$\alpha_{i,j} + \alpha_{j,i} + \beta_{i,j} + \beta_{j,i} \geq 1 \quad \forall i \neq j$$

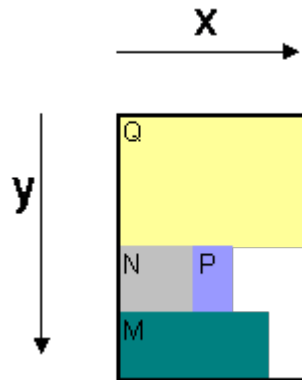
$$\alpha_{i,j}, \beta_{i,j}, \gamma_i = \{0,1\}; x_i, y_i, r_i, s_i \geq 0$$

Sin que sirva de precedente se incorpora un ejemplo numérico)

$$DX=5 \quad DY=8$$

Pieza M: (4,2) ; Pieza N: (2,2) ; Pieza P: (2,1) ; Pieza Q: (4,5)

Una posible solución gráfica al problema sería



La solución anterior se puede definir del siguiente modo: $(Q1, N0, P1, M0)$, que representa que primero se ubica la pieza Q en posición perpendicular a la definida, luego N en su posición natural en la esquina más arriba y más a la izquierda posible. Luego el Producto P en posición perpendicular, y el producto M en posición natural.

- b) Sobre el resultado anterior. ¿Cuál es el resultado de ejecutar una inserción del 4 elemento en la segunda posición?

$(Q1, M0, N0, P1)$

- c) Sobre el resultado anterior. ¿Cuál es la solución de ejecutar una 2-permutación del primer elemento con el tercero?

$(N0, M0, Q1, P1)$

- d) ¿Cuál es la solución resultado de ejecutar un crossover (operador de los algoritmos genéticos) entre la solución $(M0, N0, P1, Q1)$ y $(M1, P1, Q0, N0)$ fijando las posiciones 2 y 3

$(M1, N0, P1, Q0)$

6.5 Quinielas

Después de mucho pensar ha llegado a la elemental conclusión de que trabajando y ahorrando nadie se hace rico.

Así pues, se ha lanzado por el terreno legal más cómodo para hacer dinero sin dar ni golpe: las apuestas. Concretamente las Quinielas, porque le permitirán hacer uso de su conocimiento exhaustivo del mundo del fútbol (tantas horas invertidas en leer el Marca más pronto o más tarde iban a dar sus frutos).

Pero su manera de ganar dinero será científica, y así tantas horas invertidas en Métodos Cuantitativos también serán útiles.

Quiere hacer un modelo matemático que rellene una columna en la quiniela con un número limitado de apuestas (todavía no es rico), maximizando la probabilidad de acertar con unas limitaciones que irá incorporando.

Sea $\alpha_{i,1}$ la variable que indica que apuesta por la victoria del equipo de casa en el partido i .

Sea $\alpha_{i,x}$ la variable que indica que apuesta por el empate en el partido i .

Sea $\alpha_{i,2}$ la variable que indica que apuesta por la victoria del equipo foráneo en el partido i .

- (1 punto) a) Modele una restricción (o juego de ellas) que asegure que apuesta en todos los partidos.
- (1 punto) b) Modele una restricción (o juego de ellas) que mostrará que no quiere apostar por menos de 4 *unos* ni por más de 7 *unos*.
- (1 punto) c) Modele que si le pone un 1 a un determinado partido A, le debe poner un 2 a un determinado partido B, aunque no necesariamente a la inversa.
- (2 puntos) d) Modele una limitación superior de 5 dobles y 2 triples.
- (1 punto) e) Establezca una restricción que limite el número total de apuestas a 2000.
- (1 punto) f) Incorpore una función objetivo que maximice la probabilidad de acertar.
- (2 puntos) g) Un análisis estadístico previo parece indicar que el ingreso esperado no crece linealmente con las apuestas jugadas sino en una curva en S con los siguientes puntos (en escala logarítmica). Establezca un objetivo lineal que pretenda maximizar el ratio beneficio esperado frente a coste de las apuestas.

Log(número de apuestas)	Log(coste de las apuestas)	log(Beneficio Esperado)
0	-0,52	0,04

1	0,48	0,30
2	1,48	1,00
3	2,48	3,00
4	3,48	4,48
5	4,48	5,30
6	5,48	5,78
6,68	6,16	5,85

- (1 punto) h) Una determinada combinación se puede representar mediante una secuencia de 14 letras donde A representa un 1, B representa una X, C representa un 2, D representa 1X, E representa 12, F representa X2 y G representa 1X2. Sea una posible solución al problema planteado ADEABCEFFBEBFB. Y sea otra posible solución al mismo problema EDABBAEGAABCFF. ¿Qué combinación obtendría aplicando el operador genético de Order Crossover (OX) o Cruce, con el corte en las posiciones 5 y 9?

Notas de apoyo.

Como sabe la Quiniela es un juego donde se deben acertar el máximo número de resultados de 14 partidos de fútbol (por el momento prescindiremos del 15). El resultado puede ser 1, X o 2. Es decir gana el de casa, empatan o gana el de fuera.

Tras su estudio de la excelente prensa deportiva y el análisis de vídeos, enfermerías, árbitros y previsiones climatológicas, usted establece una probabilidad de que en cada uno de los partidos se produzca uno de los resultados ($P_{i,1}$ es la probabilidad de que en el partido i gane el equipo de casa, $P_{i,X}$ es la probabilidad de que en el partido i empaten y $P_{i,2}$ es la probabilidad de que en el partido i gane el equipo visitante, evidentemente $P_{i,1} + P_{i,X} + P_{i,2} = 1$).

Tenga en cuenta para el punto f) que en cada partido la probabilidad de acertar es la suma de las probabilidades de los signos jugados. Y además que la probabilidad de acertar la quiniela es la multiplicación de probabilidades. Y además, si X_i es la probabilidad de acertar el partido i

$$\text{MAX} \prod_i X_i \text{ es equivalente a } \text{MAX} \sum \log(X_i)$$

Además la función logaritmo se puede aproximar por este conjunto de puntos:

X	log(X)
---	--------

0,01	-2
0,1	-1
0,3	-0,52
1	0

El número de apuestas jugadas es la multiplicación para todos los partidos del número de columnas jugadas en cada partido. Así, si para un partido jugamos un doble entonces el número de apuestas se multiplica por 2, y si jugamos un triple se multiplica por 3. Si el partido se juega a simple el número de apuestas se multiplica por 1.

Recuerde que $\log(A*B)=\log(A)*\log(B)$ y $\log(A/B)=\log(A) - \log(B)$

$\log(1)=0$; $\log(2)= 0,30$; $\log(3)=0,48$

Resolución

a) Modele una restricción (o juego de ellas) que asegure que apuesta en todos los partidos.

$$\alpha_{i,1} + \alpha_{i,X} + \alpha_{i,2} \geq 1$$

b) Modele una restricción (o juego de ellas) que mostrará que no quiere apostar por menos de 4 unos ni por más de 7 unos.

$$\sum_i \alpha_{i,1} \leq 7$$

$$\sum_i \alpha_{i,1} \geq 4$$

c) Modele que si le pone un 1 a un determinado partido A, le debe poner un 2 a un determinado partido B, aunque no necesariamente a la inversa.

$$\alpha_{A,1} = 1 \rightarrow \alpha_{B,2} = 1$$

$$\alpha_{B,2} \geq \alpha_{A,1}$$

d) Modele una limitación superior de 5 dobles y 2 triples.

Definimos 3 variables binarias para cada partido (δ , ε , ϕ) que indicarán si es simple doble o triple.

$$y_i = \alpha_{i,1} + \alpha_{i,X} + \alpha_{i,2}$$

$$y_i = \delta_i + 2\varepsilon_i + 3\phi_i$$

$$1 = \delta_i + \varepsilon_i + \phi_i$$

$$\sum_i \phi_i \leq 2$$

$$\sum_i \varepsilon_i \leq 5$$

e) Establezca una restricción que limite el número total de apuestas a 2000.

Definimos 3 variables binarias para cada partido (δ , ε , ϕ) que indicarán si es simple doble o triple.

$$y_i = \alpha_{i,1} + \alpha_{i,X} + \alpha_{i,2}$$

$$y_i = \delta_i + 2\varepsilon_i + 3\phi_i$$

$$1 = \delta_i + \varepsilon_i + \phi_i$$

$$\text{numerodeapuestas} = 2^{\text{numdobles}} * 3^{\text{numtriples}} \leq 2000$$

$$\log 2 \sum_i \varepsilon_i + \log 3 \sum_i \phi_i \leq \log 2000$$

f) Incorpore una función objetivo que maximice la probabilidad de acertar.

Sea X_i la probabilidad de acertar un determinado partido

$$X_i = \sum_j P_{i,j} \alpha_{i,j}$$

Sea V_i una variable equivalente a $\log(X_i)$

$$\text{MAX} \prod_i X_i \text{ es equivalente a } \text{MAX} \sum \log(X_i)$$

$$\text{MAX} \sum V_i$$

$$X_i = 0,01\lambda_{1,i} + 0,1\lambda_{2,i} + 0,3\lambda_{3,i} + \lambda_{4,i}$$

$$V_i = -2\lambda_{1,i} - \lambda_{2,i} - 0,52\lambda_{3,i}$$

$$\lambda_{1,i} + \lambda_{2,i} + \lambda_{3,i} + \lambda_{4,i} = 1$$

g) Un análisis estadístico previo parece indicar que el ingreso esperado no crece linealmente con las apuestas jugadas sino en una curva en S en escala logarítmica. Establezca un objetivo lineal que pretenda maximizar el ratio beneficio esperado frente al coste de las apuestas.

$$\begin{aligned} \text{MAX} \left(\frac{\text{BENEFICIO}}{\text{COSTE}} \right) &\equiv \text{MAX} \log \left(\frac{\text{BENEFICIO}}{\text{COSTE}} \right) \equiv \\ &\equiv \text{MAX} \log(\text{Beneficio}) - \log(\text{Coste}) \end{aligned}$$

Sea A el logaritmo del número de apuestas, sea C el logaritmo del coste de las apuestas y sea B el logaritmo del Beneficio esperado

$$\text{Max}(B - C)$$

$$B = 0,04\varphi_1 + 0,30\varphi_2 + \varphi_3 + 3\varphi_4 + 4,48\varphi_5 + 5,30\varphi_6 + 5,78\varphi_7 + 5,85\varphi_8$$

$$C = -0,52\varphi_1 + 0,48\varphi_2 + 1,48\varphi_3 + 2,48\varphi_4 + 3,48\varphi_5 + 4,48\varphi_6 + 5,48\varphi_7 + 6,16\varphi_8$$

$$A = \log 2 \sum_i \varepsilon_i + \log 3 \sum_i \phi_i$$

$$y_i = \alpha_{i,1} + \alpha_{i,X} + \alpha_{i,2}$$

$$y_i = \delta_i + 2\varepsilon_i + 3\phi_i$$

$$1 = \delta_i + \varepsilon_i + \phi_i$$

- h) Una determinada combinación se puede representar mediante una secuencia de 14 letras donde A representa un 1, B representa una X, C representa un 2, D representa 1X, E representa 12, F representa X2 y G representa 1X2. Sea una posible solución al problema planteado ADEABCEFFBEBFB. Y sea otra posible solución al mismo problema EDABBAEGAABCFF. ¿Qué combinación obtendría aplicando el operador genético de Order Crossover (OX) o Cruce, con el corte en las posiciones 5 y 9?

ADEAB**CEFF**BEBFB

EDABBAEGAABCFF

EDABBCEFFABCFF

6.6 SUDOKU

Durante este corto verano del 2005 ha triunfado como pasatiempo el denominado SUDOKU. La mayor parte de los periódicos nacionales e internacionales llevaban diariamente uno (o más) SUDOKU's, que enganchaban (o del que se decían enganchados) cientos de bañistas en cualquiera de las playas de esta nuestra comunidad.

Pocos de estos bañistas sabían que el SUDOKU es un clásico problema matemático heredero del "cuadrado mágico" y del "cuadrado latino". Los cuadrados mágicos se remontan al 2800 AC en China, aunque posteriormente aparecieron también el Antiguo Egipto y otros lugares. Consiste en definir los valores de una matriz NxN de tal manera que ninguna de las celdas repita el valor y la suma de los valores en cada fila y en cada columna (así como los de las dos diagonales principales) valgan lo mismo. A ese valor se le denomina constante mágica.

Los cuadrados latinos sin embargo son más recientes, hubo que esperar 4600 años a que Euler los describiera. Los cuadrados latinos son matrices de NxN celdas en las que N elementos se deben

asignar a cada celda, para que, en ninguna fila y en ninguna columna, se repita el mismo elemento dos veces (son especialmente útiles en el diseño de experimentos).

El Sudoku es un cuadrado latino de 9x9 donde se deben rellenar las celdas con los dígitos del 1 al 9 y no sólo se prohíben las repeticiones en filas y columnas, sino que además se prohíbe la repetición en el interior de 9 zonas disjuntas de tamaño 3x3. Además se asignan algunos valores en algunas celdas, y de este modo se pretende que haya una y sólo una solución al problema.

Por su configuración el Sudoku es también un problema de coloreado de grafos, donde cada celda es un nodo, que está unido con todos los nodos de su línea, de su columna y de su cuadrado.

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

Se pide:

- (3 puntos) Plantear un modelo matemático cuya solución sea una solución de un cuadrado mágico de 3x3 con constante mágica tan pequeña como sea posible.
- (2 puntos) Plantear un modelo matemático para el problema de un cuadrado latino de 4x4. (Atención : Este es un problema sin objetivo)
- (1 punto) El número de cuadrados latinos de orden 4 es muy elevado, nos interesa especialmente aquel que tiene los valores pequeños tan cerca de la celda superior izquierda como sea posible. Defina el objetivo del nuevo modelo.
- (2 puntos) Plantear un modelo matemático cuya solución sea una solución del SUDOKU. Así ahorraremos mucho tiempo a los bañistas que podrán dedicarse al clásico autodefinido. (Nuevamente este modelo no tendrá objetivo, sólo restricciones).

Dado que no todos los bañistas disponen de CPLEX, plantear un procedimiento de ramificación y corte que permita obtener una solución. El procedimiento debe ser lo más eficaz posible. Ramifica en cada nodo teniendo en cuenta los valores que puede asignar a cada celda, y elimina el nodo si al explotarlo no hay ninguna posibilidad.

- (1 punto) Proponga un método para seleccionar el siguiente nodo a explotar.
- (1 punto) Proponga un método para explotar un nodo.
- (1 punto) ¿Qué información debiera tener en cuenta para que el descarte de opciones al abrir cada nuevo nodo sea lo más rápido posible? Proponga un modo de almacenar dicha información

h) (1 punto) Desarrolle el árbol para las cuatro primeras celdas que vaya a generar.

Nota: (Esta nota es útil a partir del apartado b) Un modo de garantizar que todos los elementos son distintos, y pertenecientes a un conjunto limitado de opciones, es establecer un índice para cada una de las opciones. De este modo se puede definir la opción a introducir en cada celda asignando a una variable binaria, que identifique la celda y la opción que se le ha incorporado, con el valor 1 (y 0 en caso de que no se asigne dicho valor). Así se podría definir una variable binaria $\lambda_{i,j,k}$ que valiera 1 si en la celda de coordenadas i y j se asigna el valor k .

Nota 2: Si una restricción no sabe como representarla escríbala “en letra”, recuerde que el proceso y el orden del mismo es tan (o más) importante que el resultado.

6.7 Secuenciando en la Línea de Montaje

Una línea de montaje de coches tiene en medio de su sistema un almacén formado por 4 líneas en paralelo cada una con capacidad para alrededor de 20 coches, que sirve para absorber las fluctuaciones entre la primera parte de la planta (denominada trim) y la segunda parte de la misma (denominada chasis).

Los coches entran en el almacén y son distribuidos de acuerdo a un criterio que no es relevante en este momento. Los coches de una determinada línea no pueden adelantarse entre sí, es decir si se pretende sacar el tercer coche de una línea se debe secuenciar primero las unidades por delante de él.

Tras analizar la situación descubre que la calidad de la secuencia va relacionada con un concepto que denominan “violación de restricciones”. Por lo visto hay una serie de restricciones de diferentes niveles de importancia cada una. Una restricción se puede representar del siguiente modo: “No más de dos Fort Juerga con Lavafaros de cada 6 coches”.

Cómo evalúa el cumplimiento de restricciones no es objeto del presente problema. Se admite que un evaluador actuaría del siguiente modo: cada coche $\langle i \rangle$ al ser incorporado a una secuencia S viola unos puntos que calculará una función $G(S^*\langle i \rangle)$. Y alguien ya tiene desarrollado el evaluador.

Tras explorar diferentes alternativas decide que lo mejor es buscar en cada iteración del proceso (es decir cada vez que hay que sacar un coche) la mejor secuencia posible de 10 unidades y sacar el primer coche de dicha secuencia.

En una brillante idea se le ocurre que la representación de la solución más adecuada, no es la secuencia de unidades a extraer sino la secuencia de las líneas desde las que se van a extraer las unidades. Así una solución (4,2,1,1,2,1,3,4) sacaría primero el primer coche de la línea 4 (que evidentemente sabemos cual es), luego el primer coche de la línea 2, luego el primero de la línea 1 seguido del segundo de la línea 1, posteriormente el segundo de la línea 2 seguido del tercero de la línea 1, etcétera.

Para sacar 10 coches tiene por tanto 4^{10} combinaciones y por tanto desprecian (aunque no deberían) la posibilidad de hacer una enumeración completa de las soluciones.

Se le pide que:

- Diseñe un algoritmo que genere una solución aleatoria.
- Diseñe un algoritmo heurístico que genere una solución que podamos considerar correcta.
- Diseñe un algoritmo de tipo Mejora Local.
- Diseñe un algoritmo de tipo genético.
- Modifique el algoritmo diseñado en b) teniendo en cuenta que en las líneas el número de unidades que puede haber puede ser muy bajo.

Cada apartado debe constar de dos partes. En la primera hay que describir el algoritmo, en la segunda hay que ejecutar el algoritmo poniendo un ejemplo.

Para diseñar el algoritmo hay que hacerlo de la manera más inteligible posible. Al ejecutar los algoritmos invéntese los datos que necesite. Si le hacen falta números aleatorios, aquí tiene una secuencia de números entre 0 y 99.

23	75	86	30	5	55	54	54	36	56	27	79	19	89	99	68	78	55	50	47	99
32	95	18	21	76	7	63	83	64	43	13	40	75	66	46	66	39	88	90	93	73
16	82	59	68	13	78	28	39	21	22	41	47	10	88	69	98	22	58	48	90	98
11	79	55	68	39	26	70	64	7	23	43	76	13	35	13	24	83	14	80	63	82
58	45	21	56	74	99	15	7	75	71	26	32	64	76	67	89	76	42	76	35	8
61	46	68	12	61	42	3	86	85	27	80	67	0	43	48	89	17	39	9	44	24
74	35	81	3	89	48	2	57	26	4	43	31	17	74	37	4	83	59	81	44	17
19	66	8	63	59	38	83	53	35	35	96	96	87	33	86	96	29	89	15	16	74
96	33	34	98	20	34	14	70	53	67	35	38	61	24	71	1	21	23	89	48	98

2 4 1 1 4 1 3 4 3 2

1 4 3 3 1 4 2 2 1 1

Resolución

Sea i : el índice que recorre las 10 posiciones de la secuencia a generar

Sea j : el índice que recorre las 4 líneas desde las que se pueden extraer unidades

Sea k : el índice que recorre las unidades en cada una de las líneas.

Sea $\text{rnd}()$ una función que define un número aleatorio entre 0 y 1

Sea $\text{entero}(x)$ una función que trunca un número aleatorio y se queda sólo con la parte entera.

Sea x_i la línea de la que extraer en la posición i

Sea a_{jk} el vector que establece la unidad en posición k del producto i

- Un algoritmo que genere una solución aleatoria podría tener la siguiente estructura.

For $i:=1$ to 10 do begin

$x_i = \text{entero}(\text{rnd}() * 4) + 1;$

end

Como ejemplo sean los números de dos cifras una secuencia aleatoria entre 0 y 99.

Dividiendo esos números por 100 tenemos números entre 0 y 1

La secuencia de 10 números aleatorios es: 23 75 86 30 5 55 54 54 36 56

La secuencia de 10 líneas será: 1 4 4 2 1 3 3 3 2 3

- b) Un algoritmo que genere una solución que podamos considerar correcta podría tener la siguiente estructura

for i:=1 to 10 do begin

$x_i = \text{Elegir la línea cuyo primer producto disponible dé un mejor valor de } G(S^*i)$

end

Como ejemplo y dado que aquí no hay nada aleatorio, y dado que no tenemos valores para los productos en el almacén no es posible poner nada más que una secuencia de números del 1 al 4. Por ejemplo la anterior. La secuencia de 10 líneas será: 1 4 4 2 1 3 3 3 2 3

Otro algoritmo que genere una solución que podamos considerar correcta sería:

for i:=1 to 10 do begin

$x_i = \text{Elegir la línea cuyo primer producto disponible sea el más antiguo de todos.}$

end

Otro algoritmo que genere una solución que podamos considerar correcta sería:

for i:=1 to 10 do begin

$x_i = (i \bmod 4) + 1;$

end

Este algoritmo daría como resultado 1,2,3,4,1,2,3,4,1,2 puesto La función mod es la función resto.

Otro algoritmo que genere una solución que podamos considerar correcta sería:

for i:=1 to 10 do begin

$x_i = \text{La línea con menor número de coches;}$

end

- c) Un algoritmo de Mejora local exige definir un vecindario y un procedimiento de descenso. Un vecindario podría ser (poniendo un ejemplo diferente de los tradicionales) modificar el valor de uno de los elementos del vector solución. Por ejemplo en la solución anterior 1 4 4 2 1 3 3 3 2 3 posibles vecinos serían

2 4 4 2 1 3 3 3 2 3

3 4 4 2 1 3 3 3 2 3

4 4 4 2 1 3 3 3 2 3

1 1 4 2 1 3 3 3 2 3

1 2 4 2 1 3 3 3 2 3

1 3 4 2 1 3 3 3 2 3

1 4 1 2 1 3 3 3 2 3

Denominemos a este método ModificarGen(i,j) que significa que en la posición i probamos el efecto de la extracción de un unidad de la fila j

Y el mecanismo de descenso puede ser el mecanismo de descenso rápido. Es decir en el momento que se encuentre algo mejor se sustituye

Una posible estructura del algoritmo sería:

```
generarSolucionVecinoMasCercano;
coste:=evaluarSolución(v);
i:=1;
while (i<=10) do begin
  j:=i;
  HayMejora:=false;
  while ((j<=4) and not(HayMejora)) do begin
    j:=j+1;
    aux:=x(i)
    ModificarGen(i,j);
    if evaluarSolucion(v)<=coste then HayMejora:=true else mutaciónInversa(i,aux);
  end;
  if j=4 then i:=i+1;
  if HayMejora then i:=1;
end; // del while
```

d) Un algoritmo genético podría tener la siguiente estructura

```
generarPoblacionInicial;
while not(hayQueParar) do begin
  seleccionarPadres;
  construirNuevaSolucion;
  actualizarPoblación;
  provocarMutaciones;
end;
```

Por tanto para definir el procedimiento hace falta (asumiendo que tenemos una función G que evalúa)

Definir un tamaño de población inicial: Por ejemplo 10

Definir un modo de generación de la población inicial: aleatoriamente

Definir un modo de parar: al cabo de 1000 iteraciones.

Se selecciona un modo de selección de padres en función de una ruleta donde el valor de fitness sea la inversa de la valoración de la solución.

Se define un procedimiento de cruce: en este caso se propone un OX con un punto de cruce, donde la sección del Padre y la Madre se incorporan íntegramente no haciendo falta reparación.

Se actualiza la población: Si una solución obtenida del cruce es mejor que la peor de las soluciones la sustituye.

Para provocar mutaciones cada 100 iteraciones se sustituyen los 5 peores miembros de la población por 5 soluciones aleatoriamente construidas.

Un ejemplo para lo único representable (la función de cruce). Sean dos soluciones aleatorias construidas con los 10 primeros números de las filas segunda y tercera:

2 4 1 1 4 1 3 4 3 2

1 4 3 3 1 4 2 2 1 1

Se elige un número aleatorio (por ejemplo con el primero de la cuarta fila: 11 que dividido por 100 (para que esté entre 0 y 1) y multiplicado por 10 para elegir por donde cortar nos da 1,1 es decir 2. Cruzamos la primera y la segunda solución cortando en la segunda posición y da

2 4 3 3 1 4 2 2 1 1

- e) Para modificar el algoritmo diseñado en b) teniendo en cuenta que en las líneas el número de unidades que puede haber puede ser muy bajo. Se propone lo siguiente:

for i:=1 to 10 do begin

*x_i = Elegir la línea cuyo primer producto disponible dé un mejor valor de G(S*i) comprobando siempre que haya un producto disponible en dicha línea*

end

